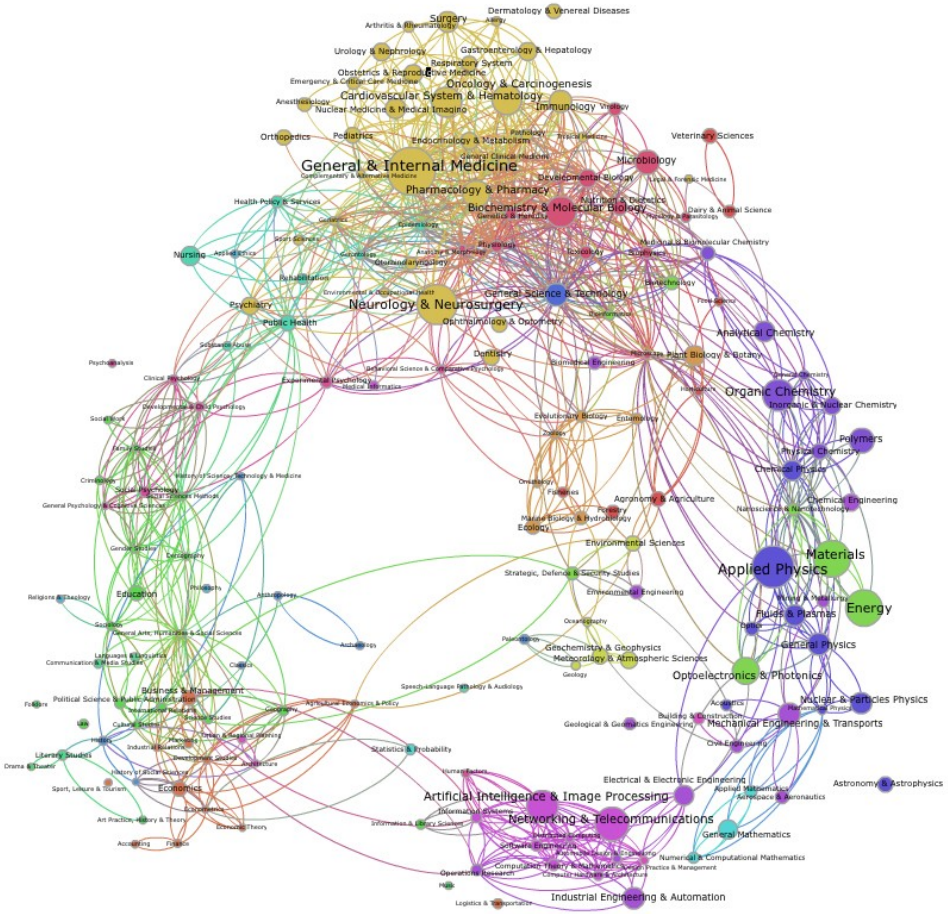
Ontology Development and Evolution



Outline

- Ontology engineering
- Ontology learning & population
- Ontology matching
- Applications of ontologies

Ontology engineering



Including slides by Valentina Tamma, A. Gomez Perez, N. Noy, D. McGuinness, E. Kendal, A. Rector and O. Corcho

www.csc.liv.ac.uk/~frank/teaching/comp08/OntologyEngineering09.ppt

9

Ontology Engineering Methodologies

York Sure, Christoph Tempich and Denny Vrandecic

9.1. INTRODUCTION

The two main drivers of practical knowledge management are technology and people, as pointed out earlier by Davenport (1996). Traditional IT-supported knowledge management applications are built around some kind of corporate or organizational memory (Abecker *et al.*, 1998). Organizational memories integrate informal, semi-formal, and formal knowledge in order to facilitate its access, sharing, and reuse by members of the organization(s) for solving their individual or collective tasks (Dieng *et al.*, 1999), for example as part of business processes (Staab and Schnurr, 2002).

The knowledge structures underlying such knowledge management systems constitute a kind of ontology (Staab and Studer, 2004) that may be built according to established methodologies (Fernandez-Lopez *et al.*, 1999; Sure, 2003). These methodologies have a centralized approach towards engineering knowledge structures requiring *knowledge engineers*, *domain experts*, and others to perform various tasks such as a *requirement analysis* and *interviews*. While the user group of such an ontology may be huge, the development itself is performed by a—comparatively—small

Ontology Engineering Methodology

York Sure[1], Steffen Staab[2], and Rudi Studer[3]

[1] SAP Research, CEC Karlsruhe, Karlsruhe, Germany, york.sure@sap.com
[2] Research Group ISWeb, University of Koblenz-Landau, Koblenz, Germany, staab@uni-koblenz.de
[3] Institute AIFB, University of Karlsruhe (TH) and FZI Research Center for Information Technology, Karlsruhe, Germany, studer@aifb.uni-karlsruhe.de

Summary. In this chapter we present a methodology for introducing and maintaining ontology based knowledge management applications into enterprises with a focus on Knowledge Processes and Knowledge Meta Processes. While the former process circles around the usage of ontologies, the latter process guides their initial set up. We illustrate our methodology by an example from a case study on skills management. The methodology serves as a scaffold for Part B "Ontology Engineering" of the handbook. It shows where more specific concerns of ontology engineering find their place and how they are related in the overall process.

1 Introduction

Ontologies constitute valuable assets that are slowly, but continuously gaining recognition and use throughout a set of disciplines – as becomes visible in Part C of this book. Ontologies frequently being a complex asset, their creation

Knowl Inf Syst (2006) 10(1): 109–131
DOI 10.1007/s10115-005-0227-4

Knowledge and Information Systems

REGULAR PAPER

Konstantinos Kotis · George A. Vouros

Human-centered ontology engineering: The HCOME methodology

Received: 7 October 2004 / Revised: 23 March 2005 / Accepted: 9 April 2005 /
Published online: 9 September 2005
© Springer-Verlag London Limited 2005

Abstract The fast emergent and continuously evolving areas of the Semantic Web and Knowledge Management make the incorporation of ontology engineering tasks in knowledge-empowered organizations and in the World Wide Web more than necessary. In such environments, the development and evolution of ontologies must be seen as a dynamic process that has to be supported through the entire ontology life cycle, resulting to living ontologies. The aim of this paper is to present the *H*uman-*C*entered *O*ntology Engineering *Me*thodology (HCOME) for the development and evaluation of living ontologies in the context of communities of knowledge workers. The methodology aims to empower knowledge workers to continuously manage their formal conceptualizations in their day-to-day activities and shape their information space by being actively involved in the ontology life cycle. The paper also demonstrates the *H*uman *C*entered *ON*tology Engineering *E*nvironment, HCONE, which can effectively support this methodology.

Keywords Semantic web · Ontology engineering · Human-centered computing · Knowledge management

DILIGENT: Towards a fine-grained methodology for DIstributed, Loosely-controlled and evolvInG Engineering of oNTologies

H.Sofia Pinto[1] and **Steffen Staab**[2] and **Christoph Tempich**[2]

Abstract.

Ontology engineering processes in truly distributed settings like the Semantic Web or global peer-to-peer systems may not be adequately supported by conventional, centralized ontology engineering methodologies. In this paper, we present our work towards the DILIGENT methodology, which is intended to support *domain experts* in a *distributed* setting to *engineer and evolve* ontologies with the help of a fine-grained methodological approach based on Rhetorical Structure Theory, viz. the DILIGENT model of ontology engineering by argumentation.

1 INTRODUCTION AND MOTIVATION

It has been a widespread conviction in knowledge engineering that methodologies for building knowledge-based systems help knowledge engineering projects to successfully reach their goals in time (cf. [12] for one of the most widely deployed methodologies). With the arrival of ontologies in knowledge-based systems the same kind of methodological achievement for structuring the ontology-engineering process has been pursued by approaches like [3, 13, 15] and their application has been proposed in such areas as the Semantic Web, too. At this point, however, we have found some mismatches between these proposals (including our own) and the requirements we meet in the Semantic Web:

1. Classical development of knowledge-based systems and of corresponding ontologies is mostly *centralized* like the targeted knowledge-based system itself. In contrast, we here consider the general tendency to support *distributed information processing with ontologies*, *e.g.* the Semantic Web, agents, web services or ontology-based peer-to-peer. Stakeholders in the ontologies being developed in these cases will hardly ever gather in one place. Yet
3. KE has mostly focused on an *up- and running system* with some moderate effort for maintenance. In contrast, ontologies for distributed information processing must permanently *evolve* in order to reflect the widely diverging needs of their users.
4. KE methodologies remain rather *coarse* and the gap between their description and concrete actions to be taken is filled by the KE. In contrast, for Semantic Web ontologies and comparable use cases, we ask the question whether we could provide the domain experts with *fine-grained* guidance in order to improve their effectiveness and efficiency in ontology engineering.

To account for some of the differences between classical knowledge engineering and ontology engineering methodologies derived from there, we have started to develop a methodology for DIstributed (cf. item 1 above), Loosely-controlled (cf. item 2) and evolvInG (cf. item 3) Engineering of oNTologies, the validity of which has been partially checked and is still being checked against experiences in two case studies (cf. [11]).

In this paper, we focus on the last consideration (cf. item 4): Could ontology engineering benefit from a more fine-grained methodological support? To answer this question, we develop a methodological hypothesis from the retrospective analysis of a historical ontology engineering task, viz. the engineering and evolution of the classification of life forms in biology. Specifically, we investigate whether some argumentation structures dominate the progress in the ontology engineering task and should therefore be accounted for in a fine-grained methodology. Then, we test the methodological hypothesis by an *in vivo* experiment of collaborative ontology engineering — once with and once without fine-grained methodological guidance.

In the remainder of this paper, we start by explaining the methodological framework of DILIGENT (Section 2). Then, we present

Chapter 2
The NeOn Methodology for Ontology Engineering

Mari Carmen Suárez-Figueroa, Asunción Gómez-Pérez, and Mariano Fernández-López

Abstract In contrast to other approaches that provide methodological guidance for ontology engineering, the NeOn Methodology does not prescribe a rigid workflow, but instead it suggests a variety of pathways for developing ontologies. The nine scenarios proposed in the methodology cover commonly occurring situations, for example, when available ontologies need to be re-engineered, aligned, modularized, localized to support different languages and cultures, and integrated with ontology design patterns and non-ontological resources, such as folksonomies or thesauri. In addition, the NeOn Methodology framework provides (a) a glossary of processes and activities involved in the development of ontologies, (b) two ontology life cycle models, and (c) a set of methodological guidelines for different processes and activities, which are described (a) functionally, in terms of goals, inputs, outputs, and relevant constraints; (b) procedurally, by means of workflow specifications; and (c) empirically, through a set of illustrative examples.

An ontology engineering methodology for DOGMA

STAR Lab Technical Report

2008

Peter Spyns, Yan Tang & Robert Meersman

affiliation:
corresponding author: Peter Spyns
keywords: ontology engineering methodology
number: STAR-2008-x1
date: 05/12/2008
status: final
reference: Journal of Applied Ontology 1-2 (3): 13 - 39



RapidOWL - an Agile Knowledge Engineering Methodology

Sören Auer
University of Leipzig
Institute for Computer Science
Augustusplatz 10-11, 04109 Leipzig, Germany
auer@informatik.uni-leipzig.de

Abstract

Agile methodologies have recently gained growing success in many economic and technical spheres. This is due to the fact that flexibility, in particular fast and efficient reactions to changed prerequisites, is becoming increasingly important in the information society. To support adaptive, semantic collaboration between domain experts and knowledge engineers, a new, agile knowledge engineering methodology, called RapidOWL is proposed. This methodology is based on the idea of iterative refinement, annotation and structuring of a knowledge base. A central paradigm for the RapidOWL methodology is the concentration on smallest possible information chunks. The collaborative aspect comes into play, when those information chunks can be selectively added, removed, annotated with comments or ratings. Design rationales for the RapidOWL methodology are to be light-weight, easy-to-implement, and support of spatially distributed and highly collaborative scenarios.

methodology (eXtreme Programming of Knowledge-based systems, [12]), which extends Extreme Programming to an agile methodology for the development of knowledge-based systems. On the other hand, RapidOWL is influenced by the Wiki idea [13], which established agile practices for collaborative text editing. However, contrary to XP.K the RapidOWL methodology stresses the generic nature of a knowledge base and thus focuses on development of knowledge bases, whose final usage scenario is either not a priori known or a single usage scenario is not easily definable. This is usually the case for conceptualizations targeting at information integration as well as for shared classification systems and vocabularies. Different from the Wiki idea on the other side RapidOWL's artifacts are structured information and knowledge represented in statements rather than the Wiki's unstructured text documents.

In this paper we will first clarify the notion of a knowledge engineering methodology (Section 2), than we present briefly approaches related to RapidOWL (Section 3). We collect some specific knowledge engineering characteristics which are, from our point of view, not adequately honored

The SWO Project: A Case Study of Applying Agile Ontology Engineering Methods in Community Driven Ontologies

Maria Copeland [1], Andy Brown [1], Helen Parkinson [2], Robert Stevens [1] and James Malone [2*]

[1]Department of Computer Science, University of Manchester, Manchester, UK
[2]European Bioinformatics Institute, Cambridge, UK

ABSTRACT

The Software Ontology Project (SWO) is a community effort to build an ontology that models software used in the generation and analysis of data for curation and preservation purposes in areas such as biomedicine. In community driven efforts, requirements are elicited from the members of these communities to help ensure the ontology is fit for purpose. This requires methods which are able to engage with users with a wide range of expertise, allow close collaboration between developers and users and that are able to respond rapidly to changing knowledge. We describe an Agile Ontology Engineering method for developing ontologies, adapted from modern Agile software engineering methods. The approach was applied within the SWO project and demonstrated promising results in engaging a diverse set of community representatives and an objective measure of ontology success, of much relevance to the active community of bio-ontology developers and users.

1 INTRODUCTION

Community driven development of ontologies have become the norm in biomedicine (Garcia *et al.*, 2010; The Gene Ontology Consortium, 2010). Engaging with a diverse set of users from a community presents challenges; some of these challenges are technical, such as when multiple ontology authors need to synchronously or asynchronously edit from multiple sites, and to allow building process. The SWO is a community driven, collaborative project with the aim of producing an ontology that captures formal descriptions of software used in the production and analysis of data for curation and preservation purproses, in order to promote standardisation and reusability of knowledge (Brown, 2011). This need is particularly pertinent in the computational biology field, where an understanding of how data were analysed is an important part of the scientific record that allows proper re-interpretation and re-use of the discipline's data.

There is a need in many Ontology Engineering efforts to engage with a community of domain experts that may not be ontology or knowledge representation experts. This presents a challenge of how to engage an audience without having to learn their domain knowledge or for them to become ontology engineers. In addition to this challenge, there are difficulties of keeping the community engaged, the ontology up-to-date and responsive to the customers. Some of these challenges in community ontology development are similar to those seen in Software Engineering.

One popular method in software engineering for tackling these issues is Agile Software Development (Martin, 2002) which is a set of software engineering methods that drives the development effort around the requirements, use cases, and continuous participation with users. Agile methods require that iterations are frequent and that software is released often with an emphasis on collaboration

Research Journal of Applied Sciences, Engineering and Technology 6(16): 2993-3000, 2013
ISSN: 2040-7459; e-ISSN: 2040-7467
© Maxwell Scientific Organization, 2013
Submitted: January 08, 2013 Accepted: January 31, 2013 Published: September 10, 2013

An Analysis of Ontology Engineering Methodologies: A Literature Review

Rizwan Iqbal, Masrah Azrifah Azmi Murad, Aida Mustapha and Nurfadhlina Mohd Sharef
Faculty of Computer Science and Information Technology, Universiti Putra Malaysia, 43400 UPM Serdang, Selangor, Malaysia

Abstract: It is now widely accepted that ontologies play a critical role in achieving the goal of machine understandable web, also known as semantic web. In order to develop ontologies, several methodologies have been proposed during the last two decades. Despite the fact, that quite a number of ontology engineering methodologies have been proposed, still the field lacks widely accepted and mature methodologies. Most methodologies lack sufficient details of techniques and activities employed in them. However, some methodologies provide sufficient details including METHONTOLOGY. This article discusses and reports a critical analysis and comparison of these methodologies. The analysis is performed based on a criterion, derived from related literature, trends and needs which evolved over the years. The results of the analysis showed that there is no completely mature methodology. Therefore, this research may act as a preliminary guide to come with a state of art ontology engineering methodology, bridging up the existing gaps and shortfalls.

Keywords: Knowledge engineering, ontology engineering, ontologies, semantic web

…but for me the winner is…

Ontology Development 101: A Guide to Creating Your First Ontology

Natalya F. Noy and Deborah L. McGuinness
Stanford University, Stanford, CA, 94305
noy@smi.stanford.edu and dlm@ksl.stanford.edu

1 Why develop an ontology?

In recent years the development of ontologies—explicit formal specifications of the terms in the domain and relations among them (Gruber 1993)—has been moving from the realm of Artificial-Intelligence laboratories to the desktops of domain experts. Ontologies have become common on the World-Wide Web. The ontologies on the Web range from large taxonomies categorizing Web sites (such as on Yahoo!) to categorizations of products for sale and their features (such as on Amazon.com). The WWW Consortium (W3C) is developing the Resource Description Framework (Brickley and Guha 1999), a language for encoding knowledge on Web pages to make it understandable to electronic agents searching for information. The Defense Advanced Research Projects Agency (DARPA), in conjunction with the W3C, is RPA nguage (DAML) by extending RDF with more expre aimed t interaction on the Web (Hendler and McGuinness 2 sciplin ndardized ontologies that domain experts can use to ate in fields. Medicine, for example, has produced large, sta tured s SNOMED (Price and Spackman 2000) and the sen of t l Language System (Humphreys and Lindberg 1993) -purpo merging as well. For example, the United Nations Development Program and Dun & Bradstreet combined their efforts to develop the UNSPSC ontology which provides terminology for products and





Natalya F. Noy and Deborah L. McGuinness.
``Ontology Development 101: A Guide to Creating Your First Ontology''.
Stanford Knowledge Systems Laboratory Technical Report KSL-01-05
and Stanford **Medical Informatics** Technical Report SMI-2001-0880, March 2001.

To share common understanding of the structure of information among people or software agents

To enable reuse of domain knowledge

To make domain assumptions explicit

To separate domain knowledge from the operational knowledge

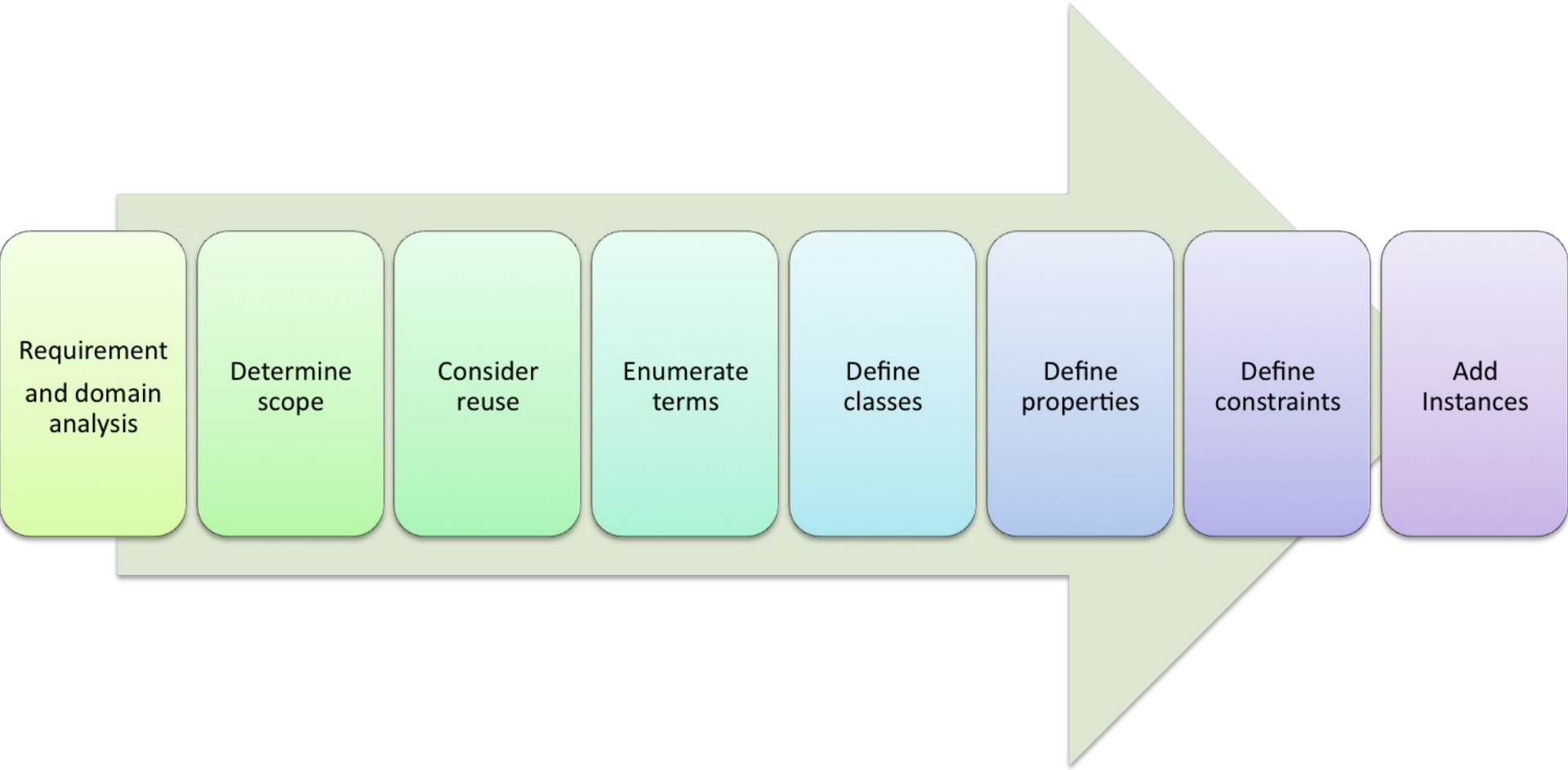
To analyze domain knowledge

available here:
https://protege.stanford.edu/publications/ontology_development/ontology101.pdf

cited by 9539 papers on Google Scholar
(up to **today**, October 21st, 2026)

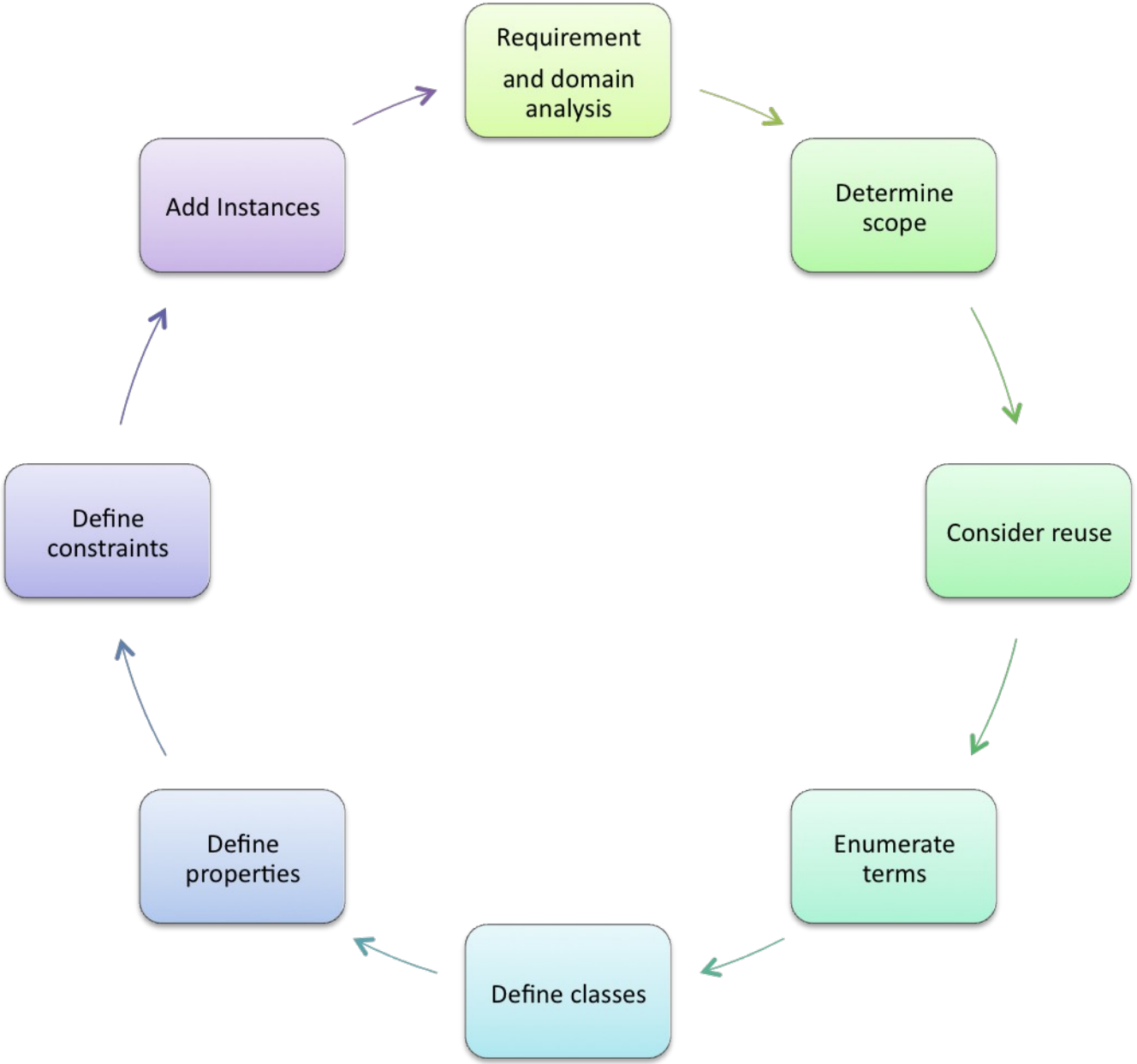
one of the oldest and simplest methodologies for developing ontologies, but still a good, flexible one

Ontology design process

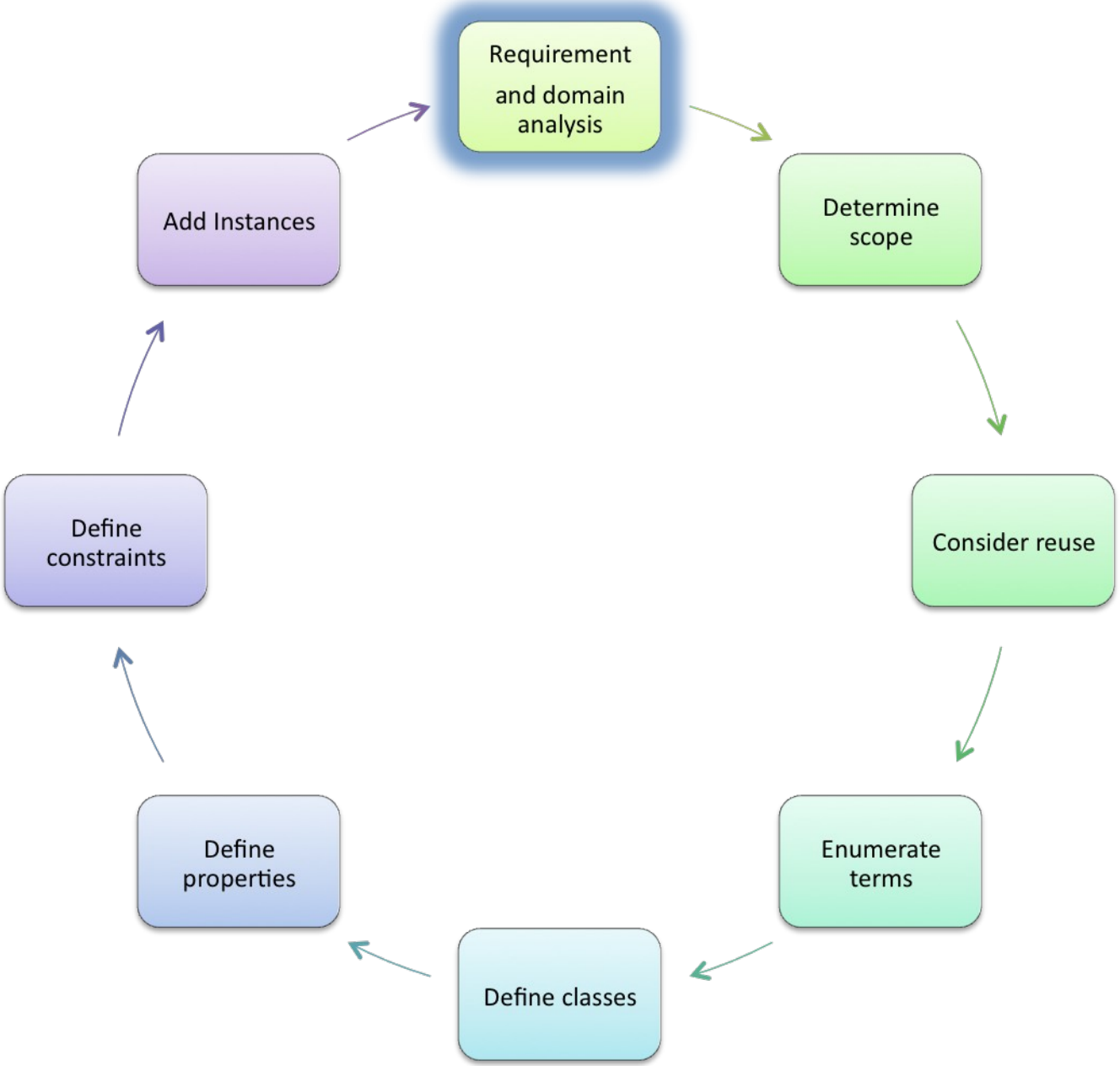


Really more like…

Ontology design process



Ontology design process

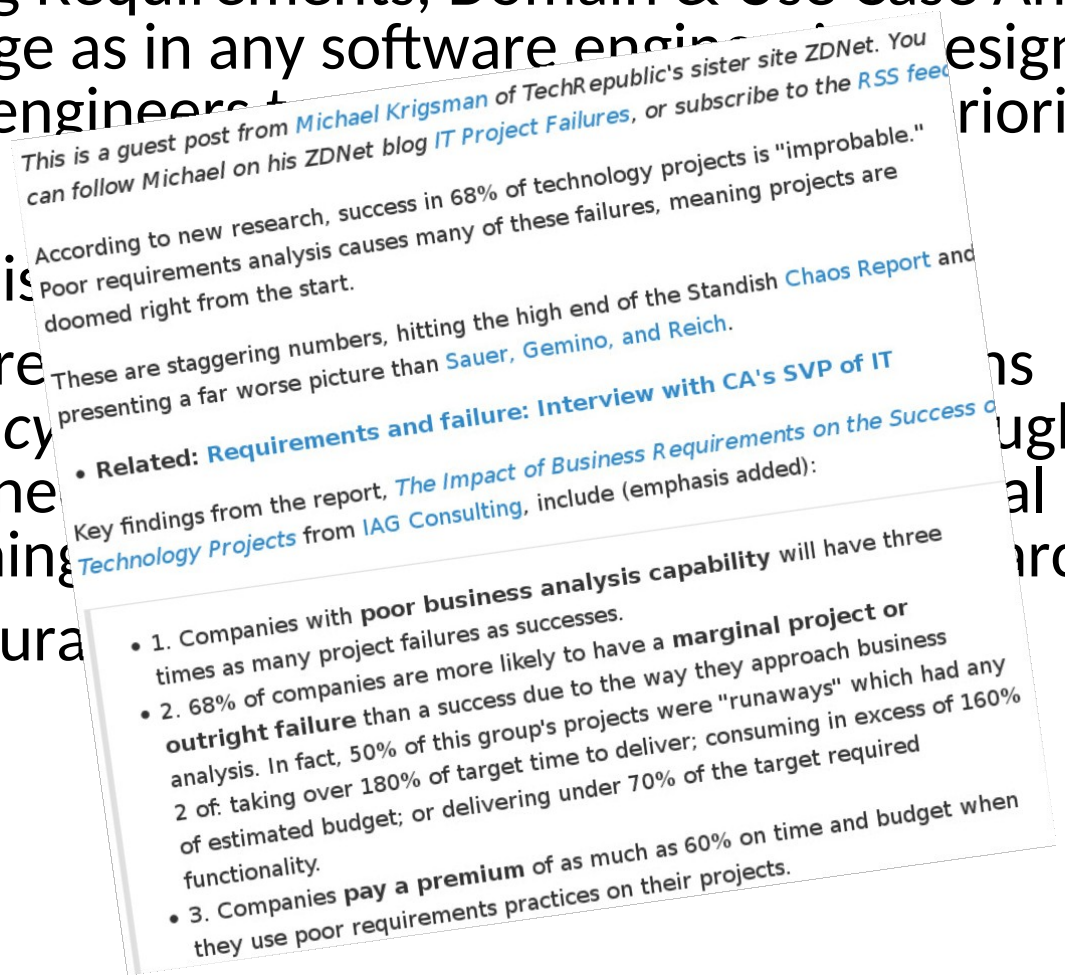


Requirement analysis

Performing Requirements, Domain & Use Case Analysis is a critical stage as in any software engi esign. It allows ontology engineers t rioritise.

The analysis

- The nature ns (*competency* ugh a reasoner) ne al for scoping and designing rchitecture;
- Architectura



This is a guest post from Michael Krigsman of TechRepublic's sister site ZDNet. You can follow Michael on his ZDNet blog IT Project Failures, or subscribe to the RSS feed

According to new research, success in 68% of technology projects is "improbable." Poor requirements analysis causes many of these failures, meaning projects are doomed right from the start.

These are staggering numbers, hitting the high end of the Standish Chaos Report and presenting a far worse picture than Sauer, Gemino, and Reich.

- **Related: Requirements and failure: Interview with CA's SVP of IT**

Key findings from the report, *The Impact of Business Requirements on the Success of Technology Projects* from IAG Consulting, include (emphasis added):

- 1. Companies with **poor business analysis capability** will have three times as many project failures as successes.
- 2. 68% of companies are more likely to have a **marginal project or outright failure** than a success due to the way they approach business analysis. In fact, 50% of this group's projects were "runaways" which had any 2 of: taking over 180% of target time to deliver; consuming in excess of 160% of estimated budget; or delivering under 70% of the target required functionality.
- 3. Companies **pay a premium** of as much as 60% on time and budget when they use poor requirements practices on their projects.

Application requirements

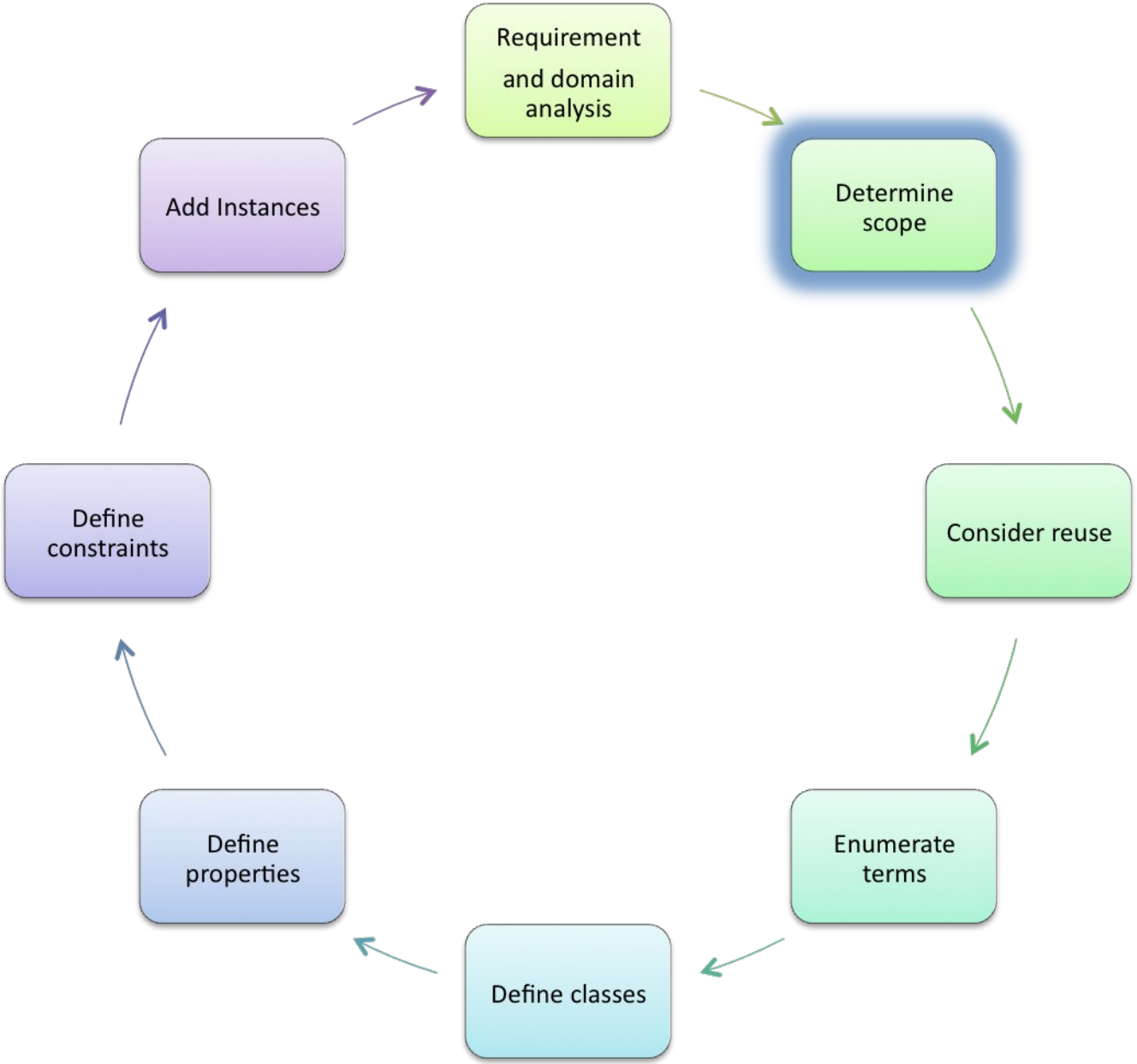
Application requirements can be acquired by:

- Identifying any controlled vocabulary used in the application;
- Identifying hierarchical or taxonomic structures intrinsic in the domain that might be used for query expansion:
  - *Vegetarian pizza such as: margherita, funghi, grilled vegetables pizza*
- Analysing structured queries and the knowledge they require
- Expressive power required: Efficient inference (requiring limited expressive power) vs. increased expressivity (requiring expensive or resource bounded computation)
- Ad-hoc reasoning to deal with particular domain requirements:
  - temporal relations, geospatial, process-specific, conditional operations
- Computational tractability
- Need for Explanations, Traces, Provenance

Domain requirements

- Take into account heterogeneity, distribution, and autonomy needs
  - software agents based applications;
- Open vs. Closed World (does lack of information imply negative information?)
- Static vs dynamic ontology processes:
  - Evolution, alignment
- Limited or incomplete knowledge
- Knowledge evolution over time
- Analysis and consistency checking of instance data
- Use Case analysis should facilitate the understanding of:
  - The information that is likely to be available
  - The questions that are likely to be asked
  - Types and roles of users

Ontology design process



Determine ontology scope

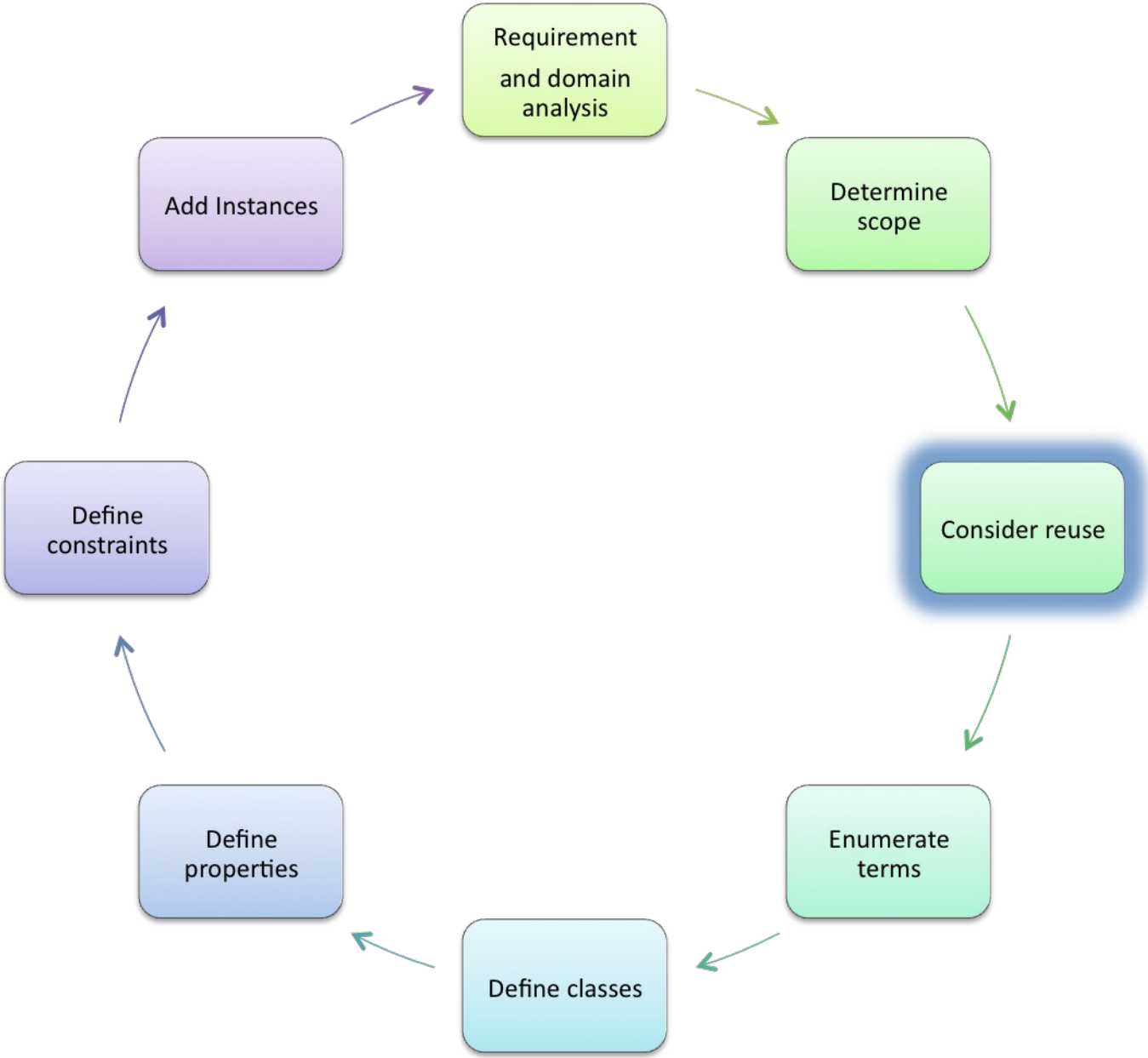
Addresses straight forward questions such as:

- What is the ontology going to be used for
- How is the ontology ultimately going to be used by the software implementation?
- What do we want the ontology to be aware of, and what is the scope of the knowledge we want to have in the ontology?

Competency Questions

- Which investigations were done with a high-fat-diet study?
- Which study employs microarray in combination with metabolomics technologies?
- List those studies in which the fasting phase had as duration one day.
- What is a vegetarian pizza?
- What type of wine can accompany seafood?

Ontology design process



Consider Reuse

- We rarely have to start from scratch when defining an ontology:
  - There is almost always an ontology available from a third party that provides at least a useful starting point for our own ontology
- Reuse allows to:
  - to save the effort
  - to interact with the tools that use other ontologies
  - to use ontologies that have been validated through use in applications

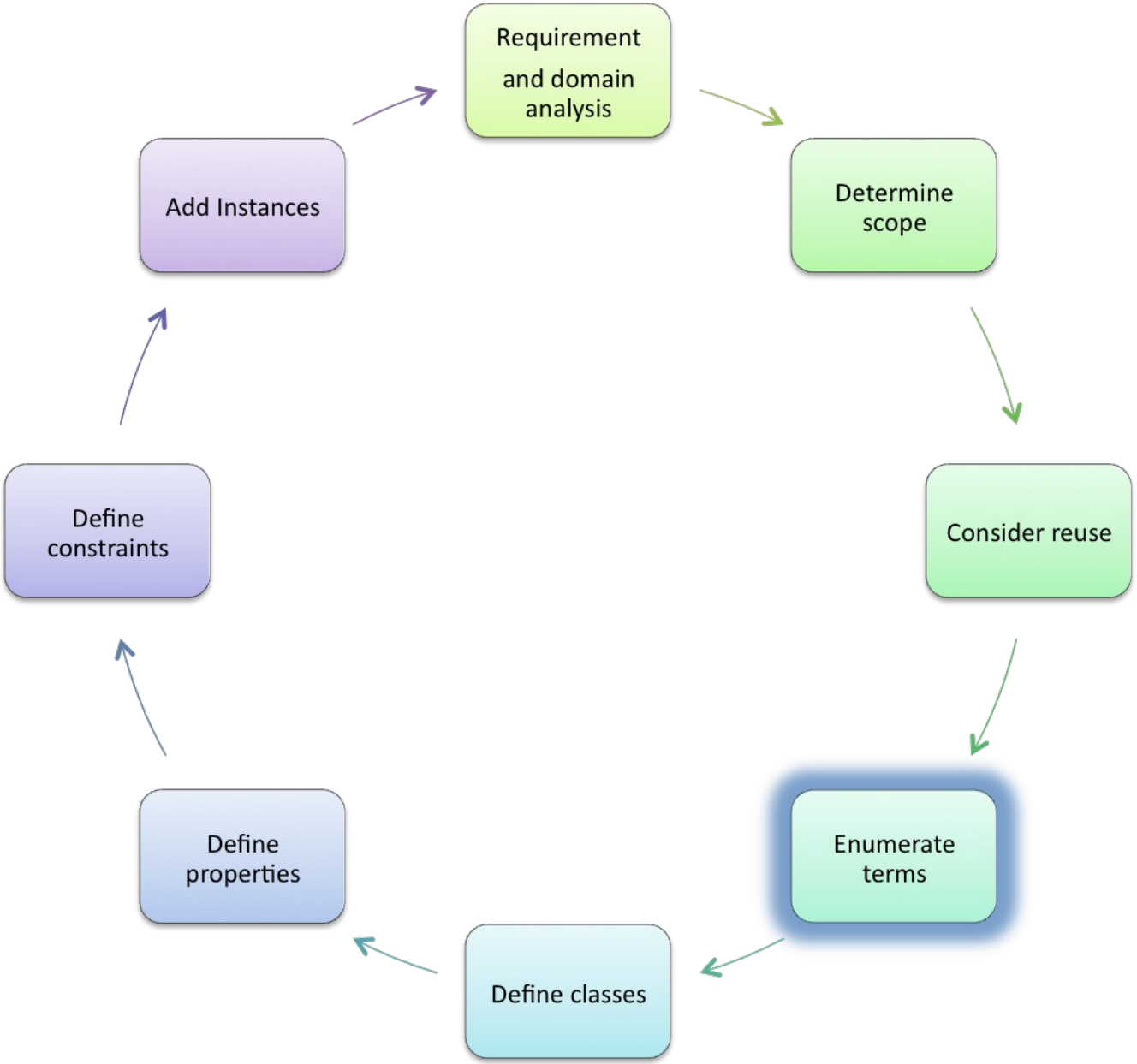
Consider Reuse

- Standard vocabularies are available for most domains, many of which are overlapping
- Identify the set that is most relevant to the problem and application issue
- A component-based approach based on modules facilitates dealing with overlapping domains:
  - Reuse an ontology module as one would reuse a software module
  - Standards; complex relationships are defined such that term usage and overlap is unambiguous and machine interpretable
- Initial brainstorming with domain experts can be highly productive; then subsequent refinement and iteration lead to the level required by the application

What to Reuse?

- Ontology libraries
  - DAML ontology library (www.daml.org/ontologies)
  - Good ontologies (https://www.w3.org/wiki/Good_Ontologies)
  - Ontologies listed by Wikipedia (https://en.wikipedia.org/wiki/Ontology_(information_science)#Published_examples)
  - Protégé ontology library (protege.stanford.edu/plugins.html)
- Upper ontologies
  - IEEE Standard Upper Ontology (suo.ieee.org)
  - Cyc (www.cyc.com)
- General ontologies
  - DMOZ (www.dmoz.org)
  - WordNet (www.cogsci.princeton.edu/~wn/)
- Domain-specific ontologies
  - UMLS Semantic Net
  - GO (Gene Ontology) (www.geneontology.org)

Ontology design process



Enumerate terms

- Write down in an unstructured list all the relevant terms that are expected to appear in the ontology
  - Nouns form the basis for class names
  - Verbs (or verb phrases) form the basis for property names
- Card sorting is often the best way:
  - Write down each concept/idea on a card
  - Organise them into piles
  - Link the piles together
  - Do it again, and again
  - Works best in a small group

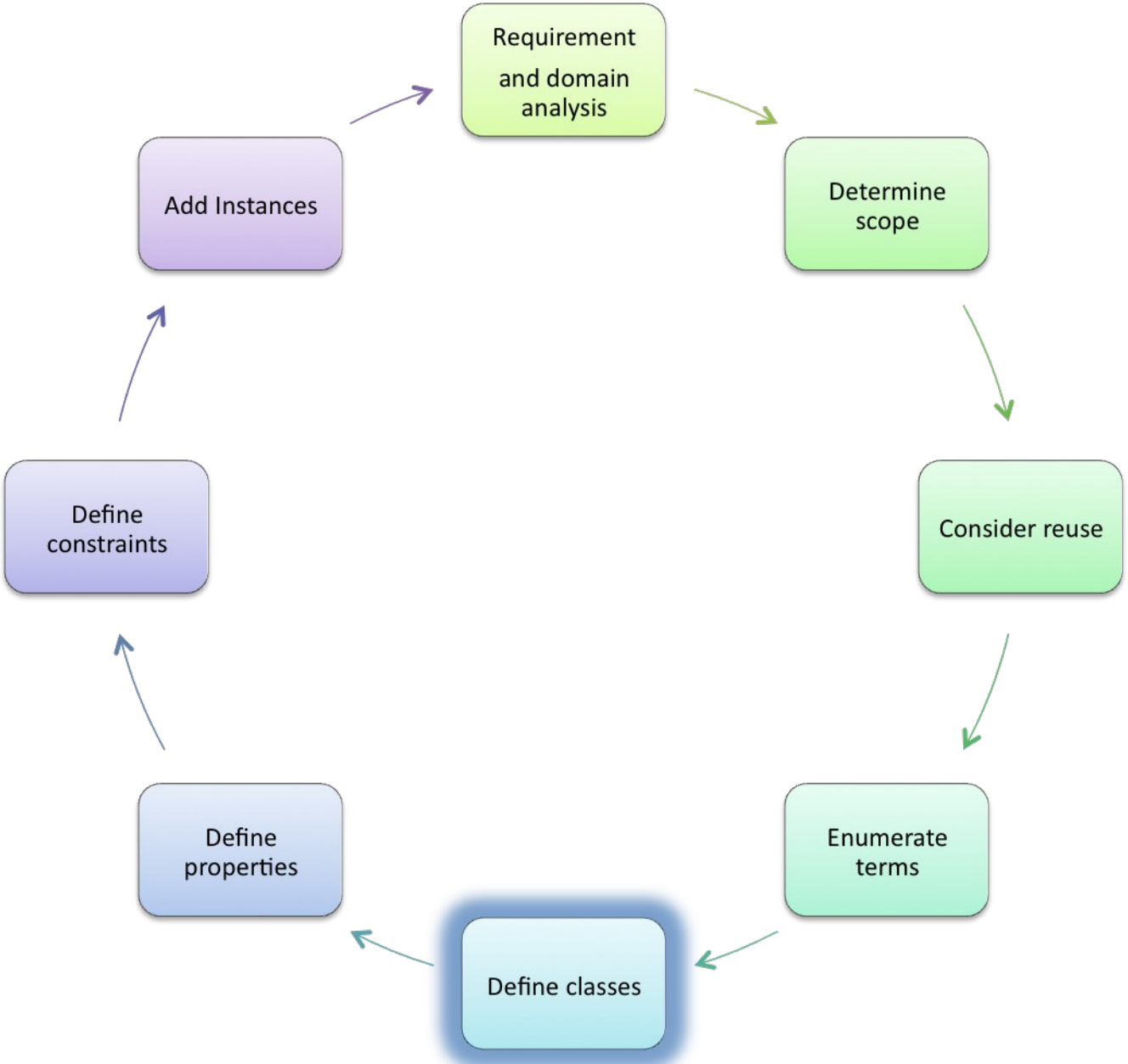
Example: animals & plants ontology

- Dog
- Cat
- Cow
- Person
- Tree
- Grass
- Herbivore
- Male
- Female

- Carnivore
- Plant
- Animal
- Fur
- Child
- Parent
- Mother
- Father

- Dangerous
- Pet
- Domestic Animal
- Farm animal
- Draft animal
- Food animal
- Fish
- Carp
- Goldfish

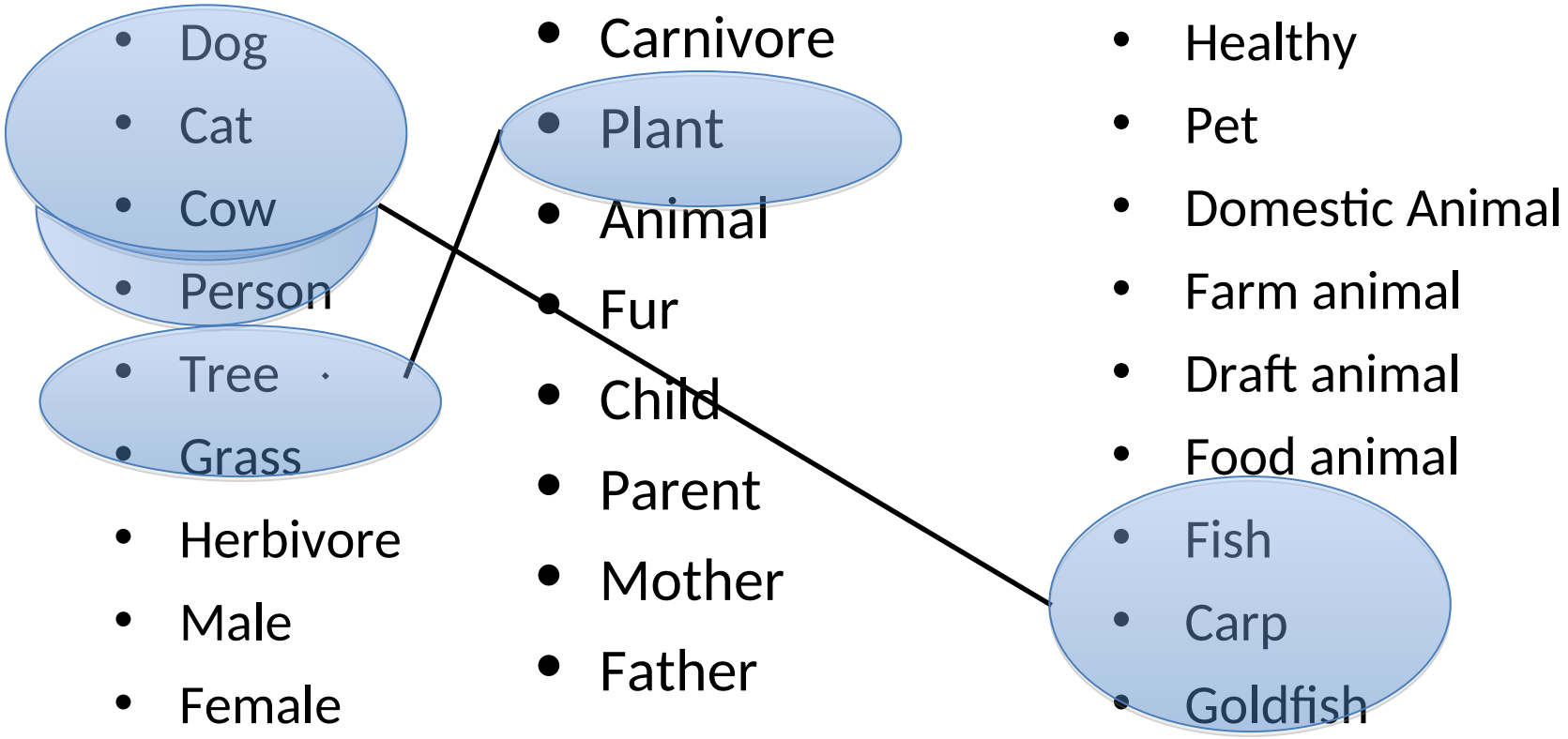
Ontology design process



Define classes and their taxonomy

- A class is a concept in the domain:
  - Animal (cow, cat, fish)
  - A class of properties (father, mother)
- A class is a collection of elements with similar properties
- A class contains necessary conditions for membership (type of food, dwelling)
- Instances of classes
  - A particular farm animal, a particular person
  - Tweety the penguin

Organise the concepts
Example: Animals & Plants



Extend the concepts

- Take a group of things and ask what they have in common
  - Then what other ‘siblings’ there might be
- e.g.
  - Plant, Animal → Living Thing
    - Might add Bacteria and Fungi but not now
  - Cat, Dog, Cow, Person → Mammal
    - Others might be Goat, Sheep, Horse, Rabbit,…
  - Cow, Goat, Sheep, Horse → Hoofed animal (“Ungulate”)
    - What others are there? Do they divide amongst themselves?
  - Wild, Domestic → Domestication
    - What other states – “Feral” (domestic returned to wild)

Choose some main axes

- Add abstractions where needed
  - e.g. “Living thing”
- identify relations (this feeds into the next step)
  - e.g. “eats”, “owns”, “parent of”
- Identify definable things
  - e.g. “child”, “parent”, “Mother”, “Father”
    - Things where you can say clearly what it means
- make names explicit

Example

- Living Thing
  - Animal
    - Mammal
      - Cat
      - Dog
      - Cow
      - Person
    - Fish
      - Carp
      - Goldfish
  - Plant
    - Tree
    - Grass
    - Fruit

- Modifiers
  - domestic
    - pet
    - Farmed
      - Draft
      - Food
  - Wild
  - Health
    - healthy
    - sick
  - Sex
    - Male
    - Female
  - Age
    - Adult
    - Child

- Relations
  - eats
  - owns
  - parent-of
  - …
- Definable
  - Carinvore
  - Herbivore
  - Child
  - Parent
  - Mother
  - Father
  - Food Animal
  - Draft Animal

Identify self-standing entities

- Things that can exist on there own
  - People, animals, houses, actions, processes, …
    - Roughly nouns
- Modifiers
  - Things that modify (“inhere”) in other things
    - Roughly adjectives and adverbs

Reorganise everything but "definable" things into pure trees – these will be the "primitives"

- Self_standing
  - Living Thing
    - Animal
      - Mammal
        - » Cat
        - » Dog
        - » Cow
        - » Person
        - » Pig
      - Fish
        - » Carp
          Goldfish
    - Plant
      - Tree
      - Grass
      - Fruit

- Modifiers
  - Domestication
    - Domestic
    - Wild
  - Use
    - Draft
    - Food
    - pet
  - Risk
    - Dangerous
    - Safe
  - Sex
    - Male
    - Female
  - Age
    - Adult
    - Child

- Relations
  - eats
  - owns
  - parent-of
  - …
- Definables
  - Carnivore
  - Herbivore
  - Child
  - Parent
  - Mother
  - Father
  - Food Animal
  - Draft Animal

Comments can help to clarify

- Self_standing
  - Living Thing
    - Animal
      - Mammal
        - » Cat
        - » Dog
        - » Cow
        - » Person
        - » Pig
      - Fish
        - » Carp
        - Goldfish
    - Plant
      - Tree
      - Grass
      - Fruit

- ***Abstract ancestor concept including all living things – restrict to plants and animals for now***

Class inheritance

- Classes are organized into subclass-superclass (or generalization-specialization)

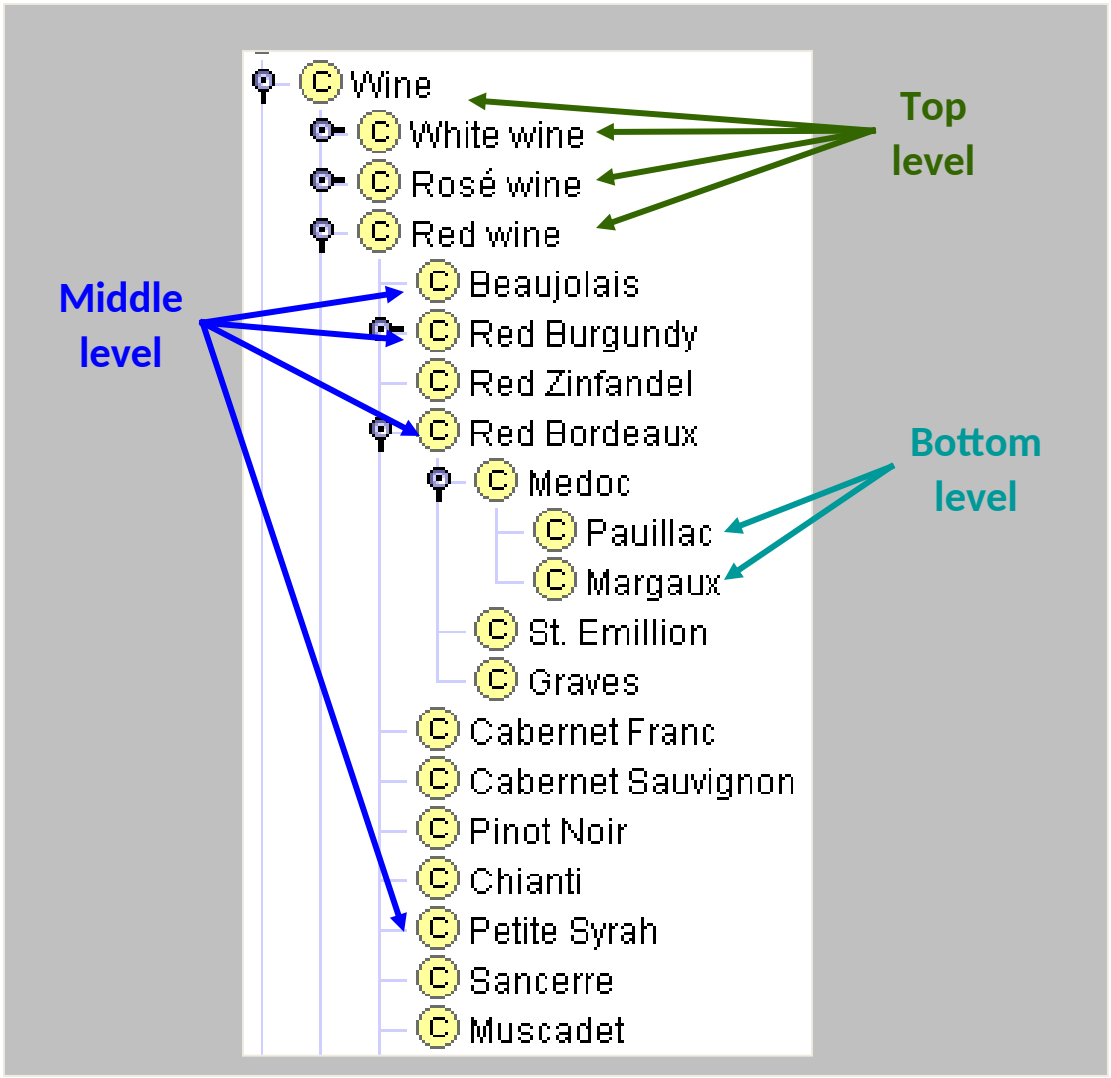
Hierarchies:

- Classes are “is-a” related if an instance of the subclass is an instance of the superclass
  - Classes may be viewed as sets
  - Subclasses of a class are comprised of a subset of the superset
- Examples
  - Mammal is a subclass of Animal
  - Every penguin is a bird or every instance of a penguin (like Tweety is an instance of bird
  - Draft animal is a subclass of Animal

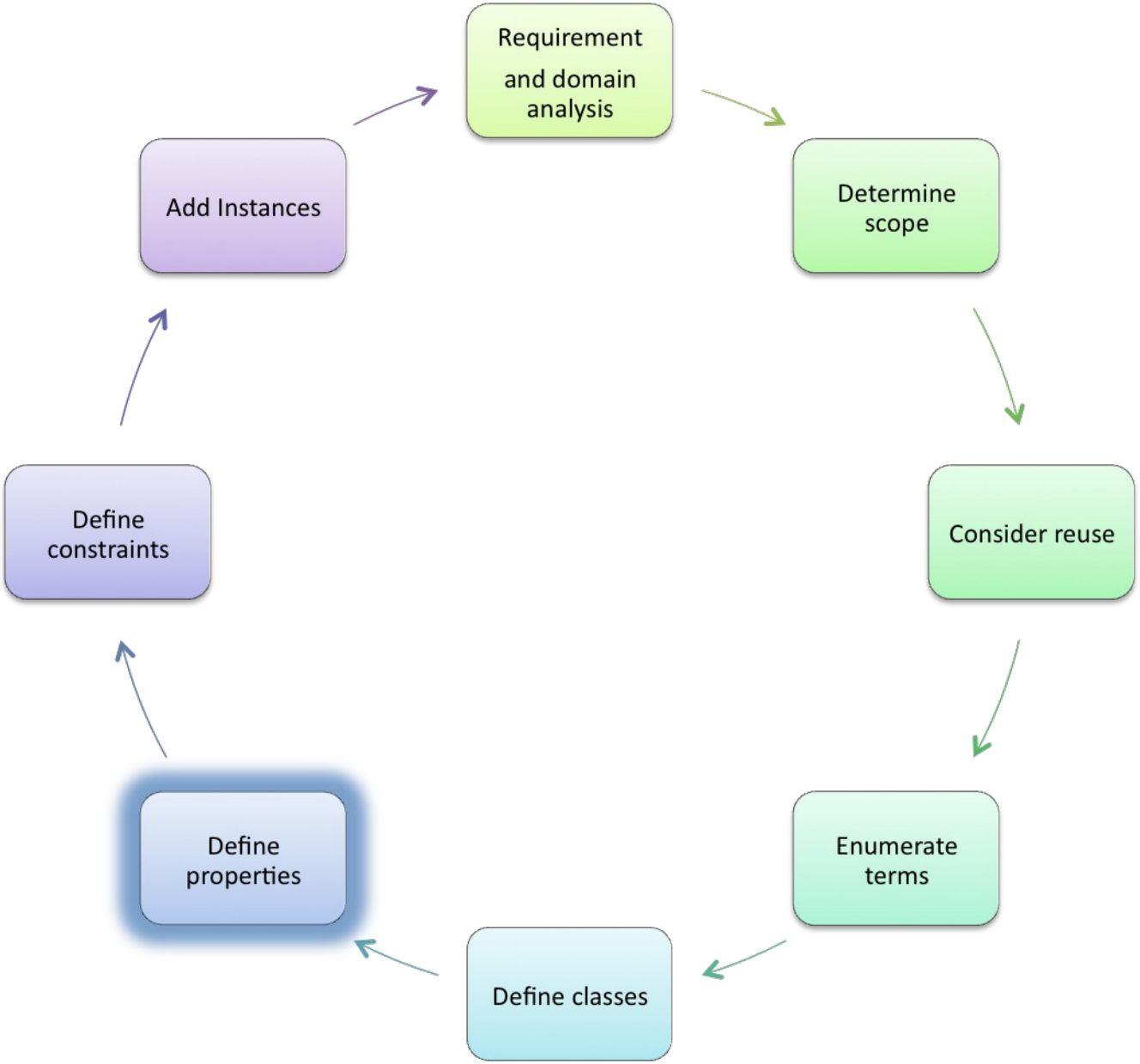
Levels in the class hierarchy

- Different modes of development
  - Top-down - define the most general concepts first and then specialize them
  - Bottom-up - define the most specific concepts and then organize them in more general classes
  - Combination (typical – breadth at the top level and depth along a few branches to test design)
- Class inheritance is Transitive
  - A is a subclass of B
  - B is a subclass of C
  - therefore A is a subclass of C

Levels in the class hierarchy



Ontology design process



Define properties

- Often interleaved with the previous step
- Properties (or *roles* in DL) describe the attributes of the members of a class
- The semantics of **subClassOf** demands that whenever A is a subclass of B, every property statement that holds for instances of B must also apply to instances of A
  - It makes sense to attach properties to the highest class in the hierarchy to which they apply

Define properties

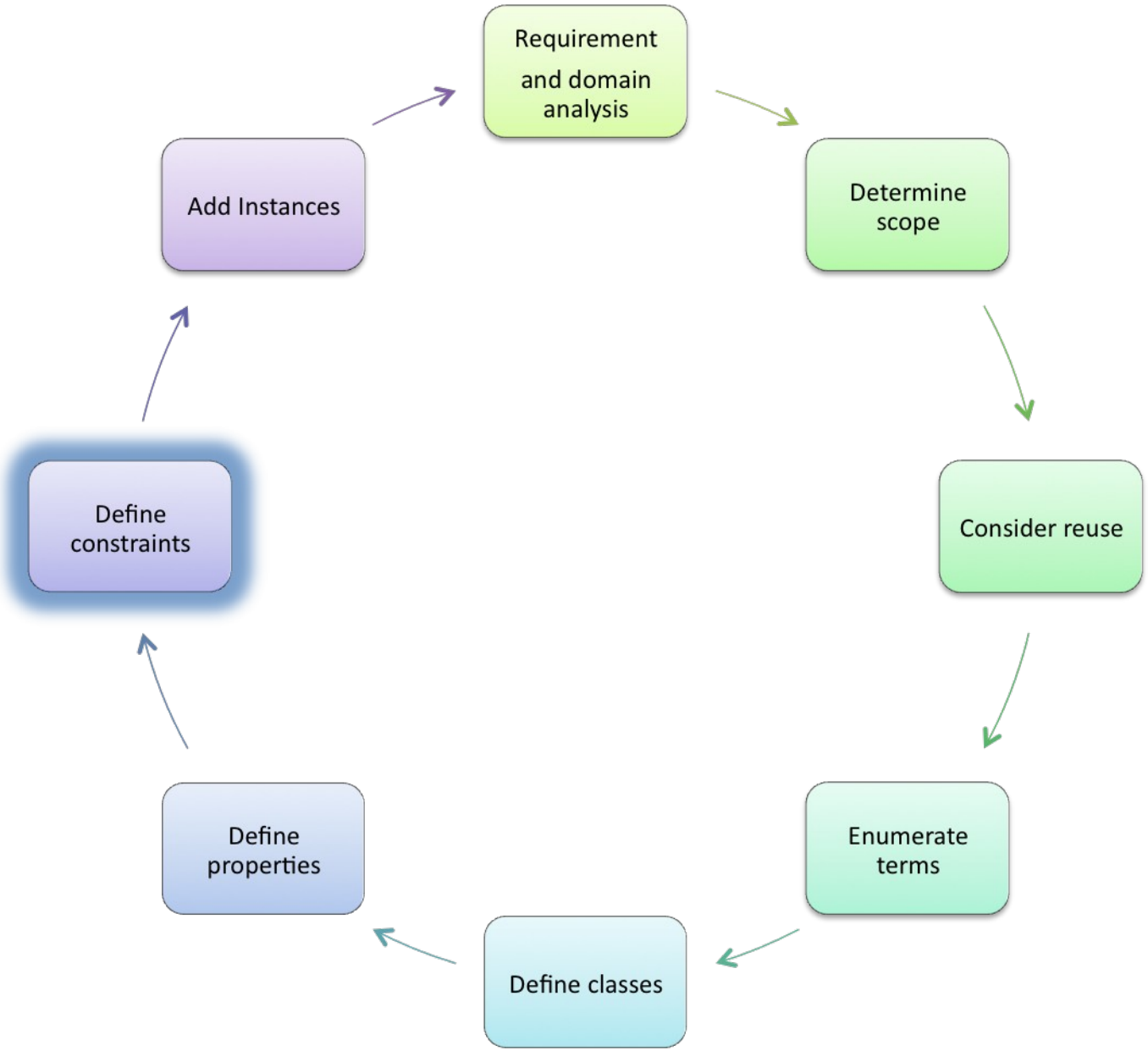
- Types of properties
  - "intrinsic" properties: flavor and color of wine
  - "extrinsic" properties: name and price of wine
  - parts: ingredients in a dish
  - relations to other objects: producer of wine (winery)
- They are represented by data and object properties
  - simple (datatype) contain primitive values (strings, numbers)
  - complex properties contain other objects (e.g., a winery instance)

Modifiers and relations

- Modifiers
  - Domestication
    - Domestic
    - Wild
  - Use
    - Draft
    - Food
    - pet
  - Risk
    - Dangerous
    - Safe
  - Sex
    - Male
    - Female
  - Age
    - Adult
    - Child

- Relations
  - eats
  - owns
  - parent-of
  - …

Ontology design process



Identify the domain and range constraints for properties

- Animal *eats* Living_thing
  - *eats* domain: Animal;
    range: Living_thing
- Person *owns* Living_thing **except** person
  - *owns* domain: Person
    range: Living_thing & not Person
- Living_thing *parent_of* Living_thing
  - *parent_of:* domain: Living_thing
    range: Living_thing

If anything is used in a special way, add a text comment

- Animal *eats* Living_thing
  - *eats* domain: Animal;
    range: Living_thing

- *ignore difference between parts of living things and living things also derived from living things*

For definable things

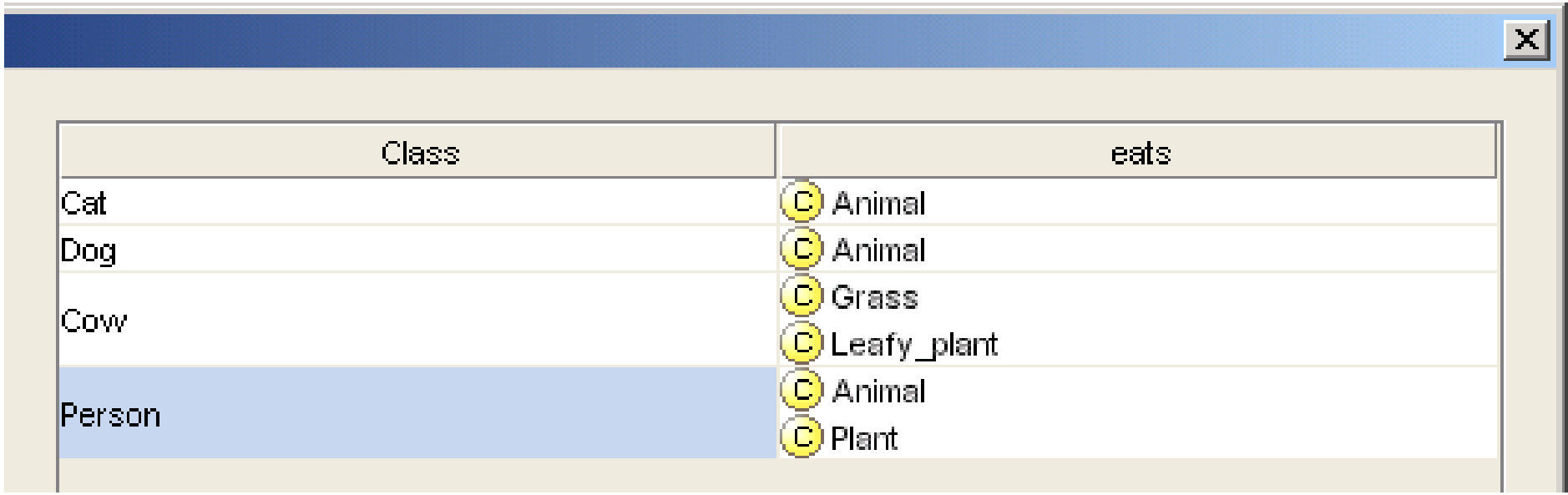
- Paraphrase and formalise the definitions in terms of the primitives, relations and other definables.
- Note any assumptions to be represented elsewhere.
  - Add as comments when implementing
- *"A 'Parent' is an animal that is the parent of some other animal" (Ignore plants for now)*
  - Parent =
    Animal and *parent_of* some Animal
- *"A 'Herbivore' is an animal that eats only plants"*
  (NB All animals eat some living thing)
  - Herbivore=
    Animal and *eats* only Plant
- "An 'omnivore' is an animal that eats both plants and animals"
  - Omnivore=
    Animal and *eats* some Animal and *eats* some Plant

Which properties can be filled in at the class level now?

- What can we say about *all* members of a class?
  - *eats*
    - *All cows eat some plants*
    - *All cats eat some animals*
    - *All pigs eat some animals & eat some plants*

Fill in the details

(can use property matrix wizard)



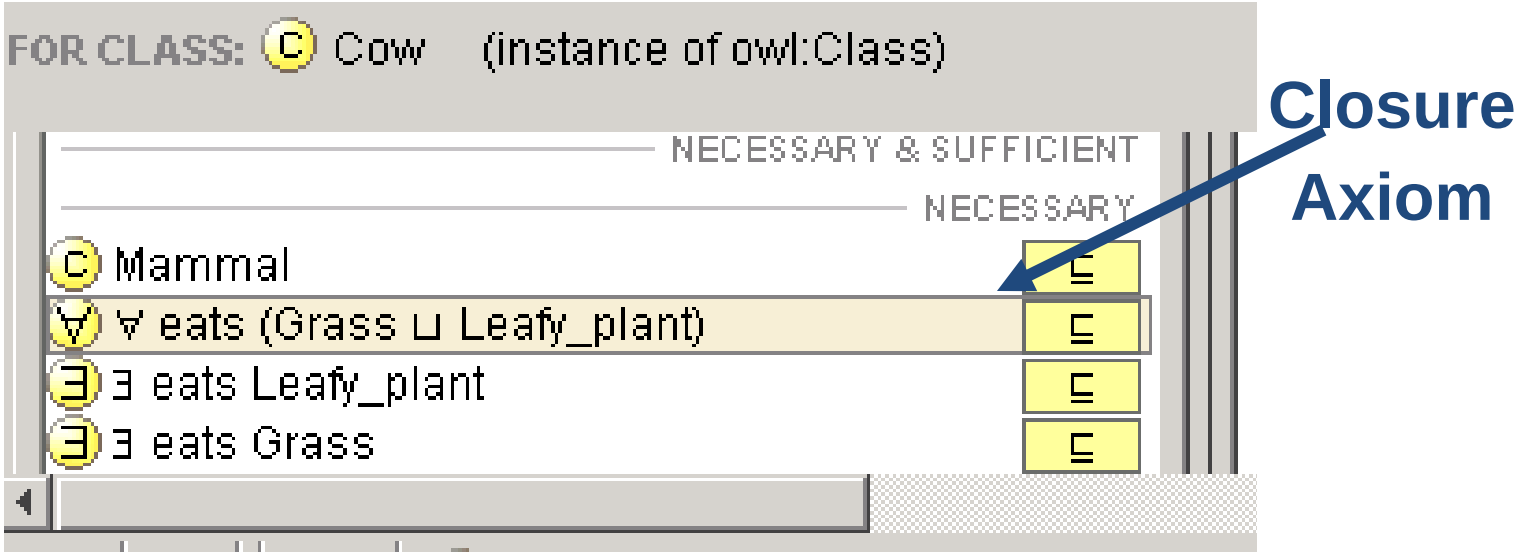
Class	eats
Cat	Animal
Dog	Animal
Cow	Grass Leafy_plant
Person	Animal Plant

Check with classifier

- Cows should be Herbivores
  - Are they? why not?
    - What have we said?
      - Cows are animals and, *amongst other things*,
        eat *some* grass and
        eat some leafy_plants
    - What do we need to say:
      Closure axiom
      - Cows are animals and, *amongst other things*,
        eat *some* plants and eat *only* plants

Closure Axiom

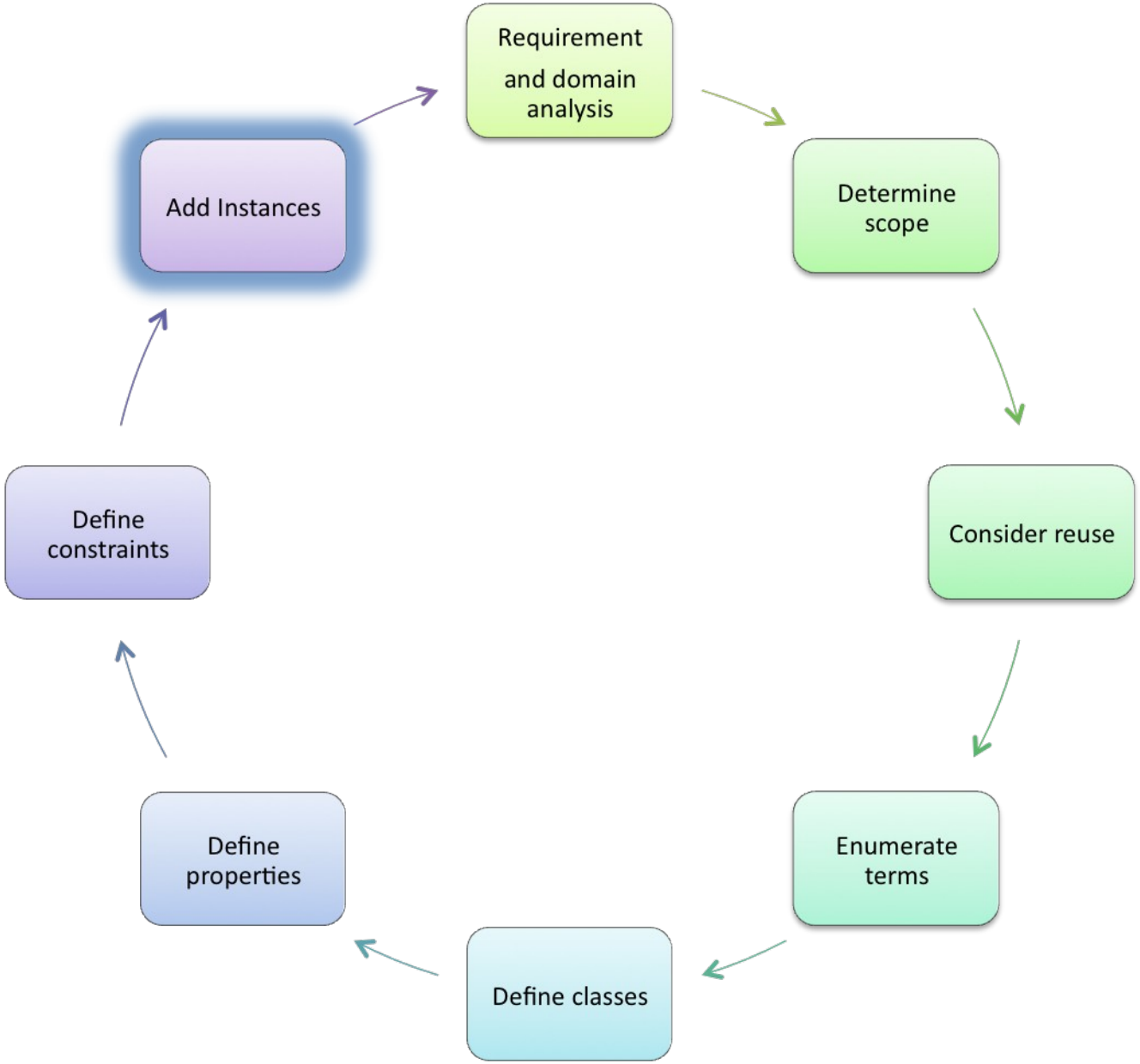
Cows are animals and, *amongst other things*,
eat *some* plants and eat *only* plants



Open vs Closed World reasoning

- **Open world reasoning**
  - Negation as contradiction
    - Anything might be true unless it can be proven false
      - Reasoning about *any world consistent with this one*
- **Closed world reasoning**
  - Negation as failure
    - Anything that cannot be found is false
      - Reasoning about *this world*
- ***Ontologies are not databases***

Ontology design process



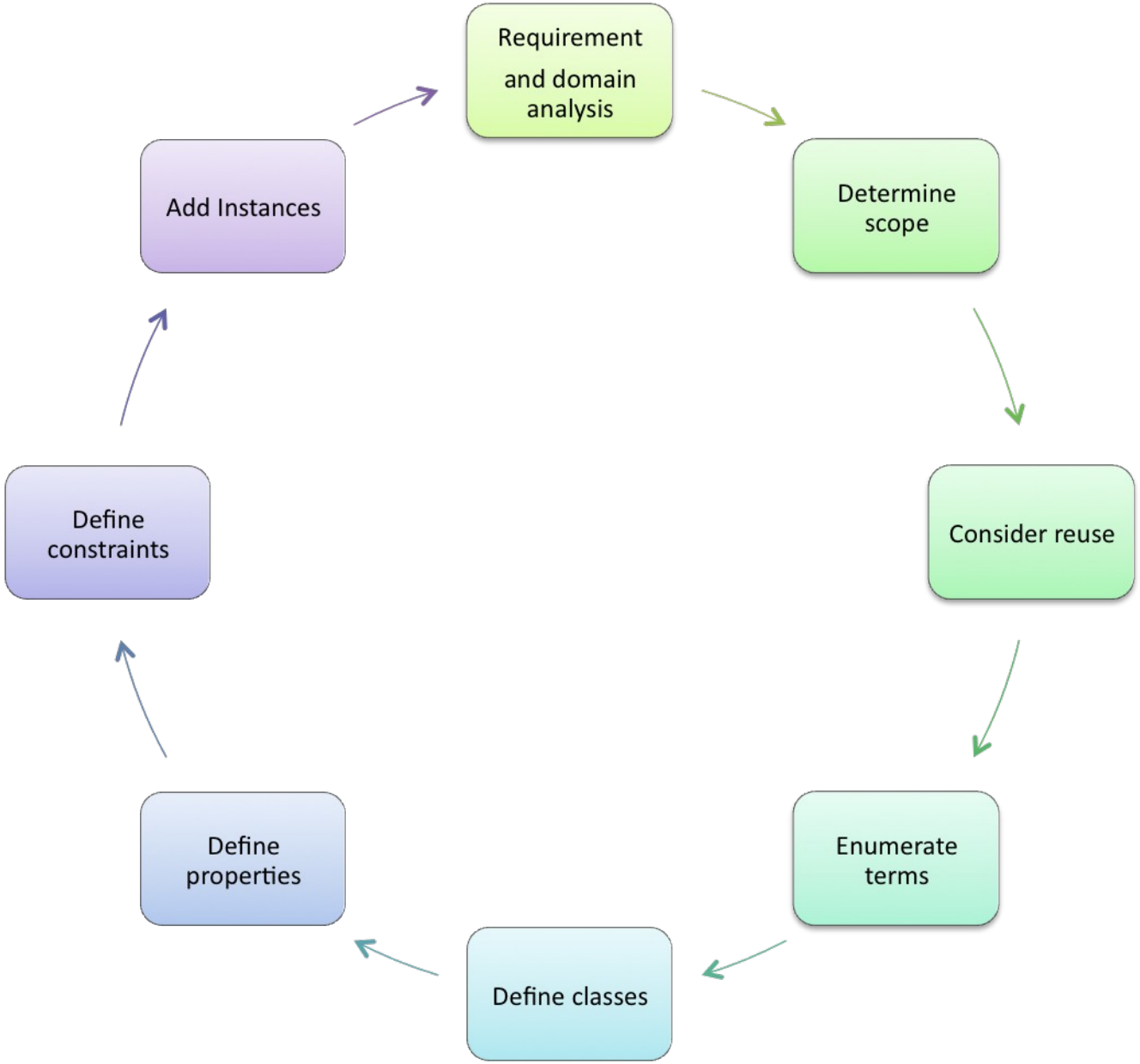
Creating instances

- Create an instance of a class
  - The class becomes a direct type of the instance
  - Any superclass of the direct type is a type of the instance
- Assign slot values for the instance frame
  - Slot values should conform to the facet constraints
  - Knowledge-acquisition tools often check that constraints are satisfied

Creating instances

- Filling the ontologies with such instances is a separate step
- Number of instances >> number of classes
- Thus populating an ontology with instances is not done manually
  - Retrieved from legacy data sources (DBs)
  - Extracted automatically from a text corpus

Ontology design process



N-ary relations

from http://www.w3.org/TR/swbp-n-aryRelations/

- In OWL a property is a ***binary*** relation: instances of properties link two individuals (or an individual and a value)
- However, sometimes the most intuitive way to represent certain concepts is to use relations to link an individual to more than just one individual or value. Such relations are called ***n-ary relations***.
- Some issues:
  - If property instances can link only two individuals, how do we deal with cases where we need to describe the instances of relations ?
  - If instances of properties can link only two individuals, how do we represent relations among more than two individuals? (***"n-ary relations"***) **Pattern 1**
  - If instances of properties can link only two individuals, how do we represent relations in which one of the participants is an ordered list of individuals rather than a single individual? **Pattern 2**

Examples

- Christine has breast tumor with high probability
  - *A relation initially thought to be binary, needs a further argument*
- Steve has temperature, which is high, but falling
  - *Two binary properties turn out to always go together and should be represented as one n-ary relation*
- John buys a "Lenny the Lion" book from books.example.com for $15 as a birthday gift
  - *From the beginning the relation is really amongst several things*
- United Airlines flight 3177 visits the following airports: LAX, DFW, and JFK
  - *One or more of the arguments is fundamentally a sequence rather than a single individual*

Can you think of some more examples?

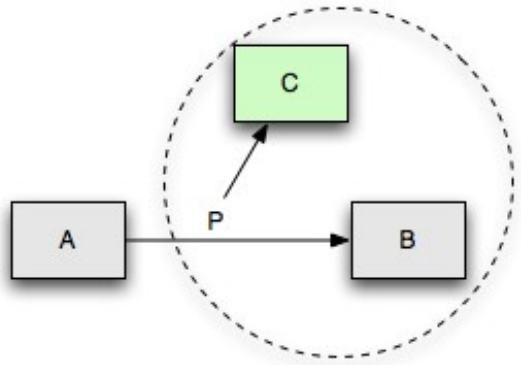
Pattern 1, N-ary relations

- Represent the relation as a class rather than a property
  - Individual instances of such classes correspond to instances of the relation
  - Additional properties provide binary links to each argument of the relation
- Basic idea: create a new class and new properties to represent an n-ary relation; then an instance of the relation linking the n individuals is then an instance of this class.
- The classes created in this way are often called "***reified relations***"

Pattern 1 case 1

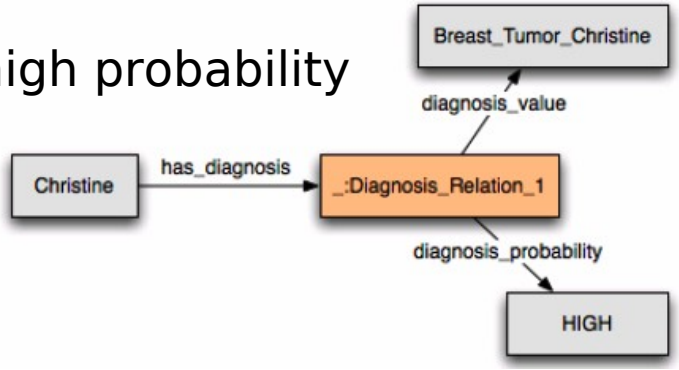
Additional attributes describing a relation:

- In this case we need to represent an additional attribute that represents a relation instance
  - Ex: *Christine has breast tumor with high probability*
- The solution is to create an individual that represents the relation instance itself, with links from the subject of the relation to this instance, and with links from this instance to all participants that represent additional information about this instance



Pattern 1, Example 1

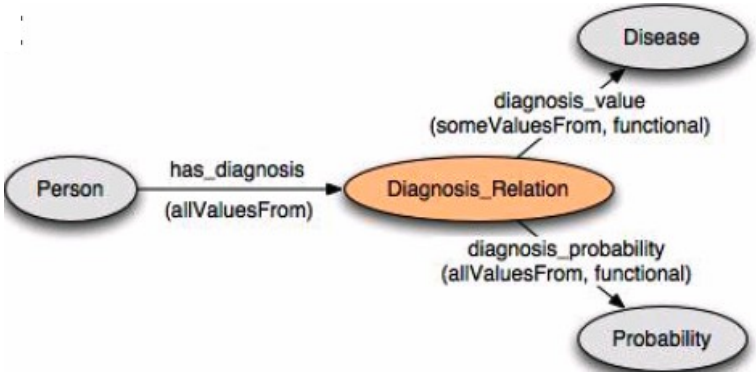
Example: Christine has breast tumor with high probability



The individual _:Diagnosis_Relation_1here represents a single object encapsulating both the diagnosis (Breast_Tumor_Christine) and the probability of the diagnosis (HIGH)

- It contains all the information held in the original 3 arguments: who is being diagnosed, what the diagnosis is, and what the probability is
- Blank nodes (rdf:Description element that does not have an rdf:about attribute assigned to it) in RDF are used to represent instances of a relation.

Class definitions:



Pattern 1 case 2

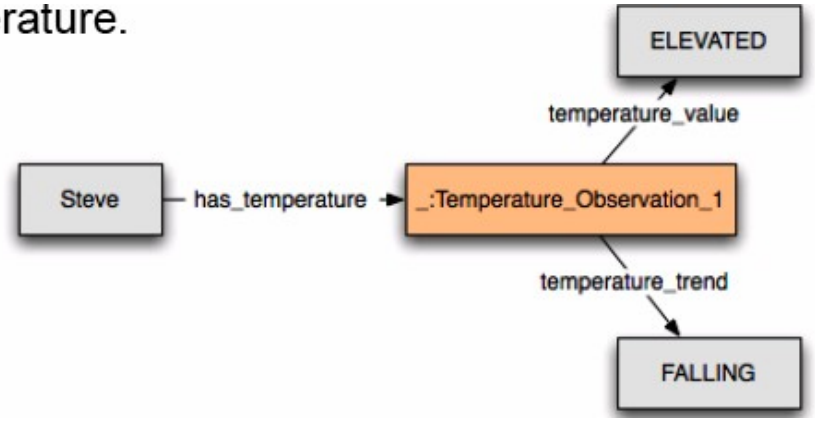
Different aspects of the same relation:

- In this case we need to represent the relation between an individual, and an object that represents different aspects of a property (relation) about the individual
  - Ex: *Steve has temperature which is high but falling*
- This instance of a relation cannot be viewed as an instance of a binary relation with additional attributes attached to it.
- It is a relation instance relating the individual and the complex object representing different facts about the specific relation between the individual and the object.

Pattern 1, Example 2

Example: *Steve has temperature, which is high, but falling*

- This cannot be viewed as an instance of a binary relation with additional attributes attached to it, but rather it is a relation instance relating the individual *Steve* and the complex object representing different facts about his temp erature.



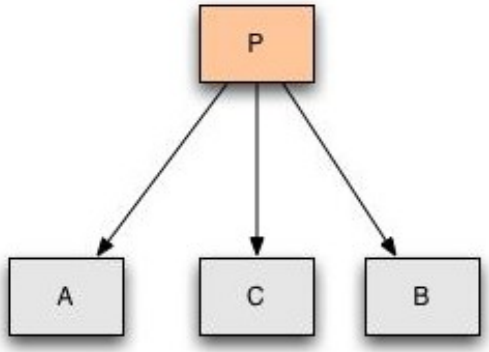
Such cases often come about in the course of evolution of an ontology when it is realized that two relations need to be collapsed.

- For example, initially, one might have had two properties (e.g. *has_temperature_level* and *has_temperature_trend*) both relating to people, and then it is realized that these properties really are inextricably intertwined because one needs to talk about "temperatures that are elevated but falling"

Pattern 1 case 3

N-ary relation with no distinguished participant:

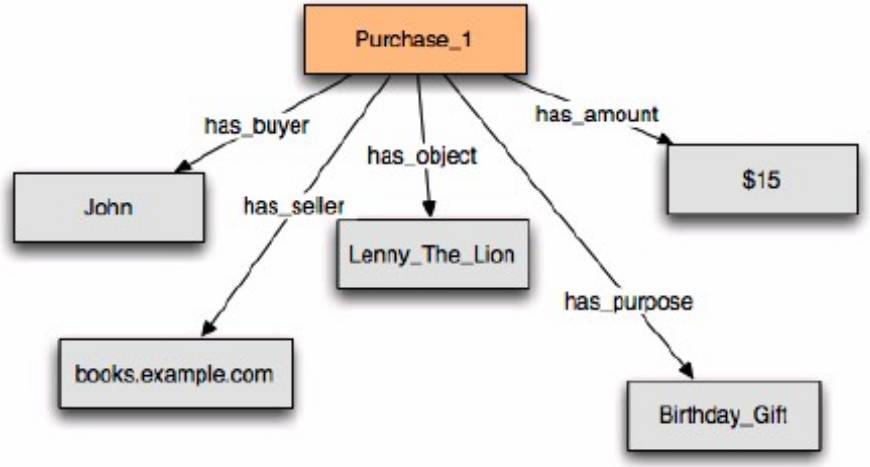
- In some cases the n-ary relationship links individuals that play different roles in a structure without any single individual standing out as the “owner” or the relation
  - Ex: *John buys a "Lenny the Lion" book from books.example.com for $15 as a birthday gift*
- The solution is to create an individual that represents the relation instance with links to all participants



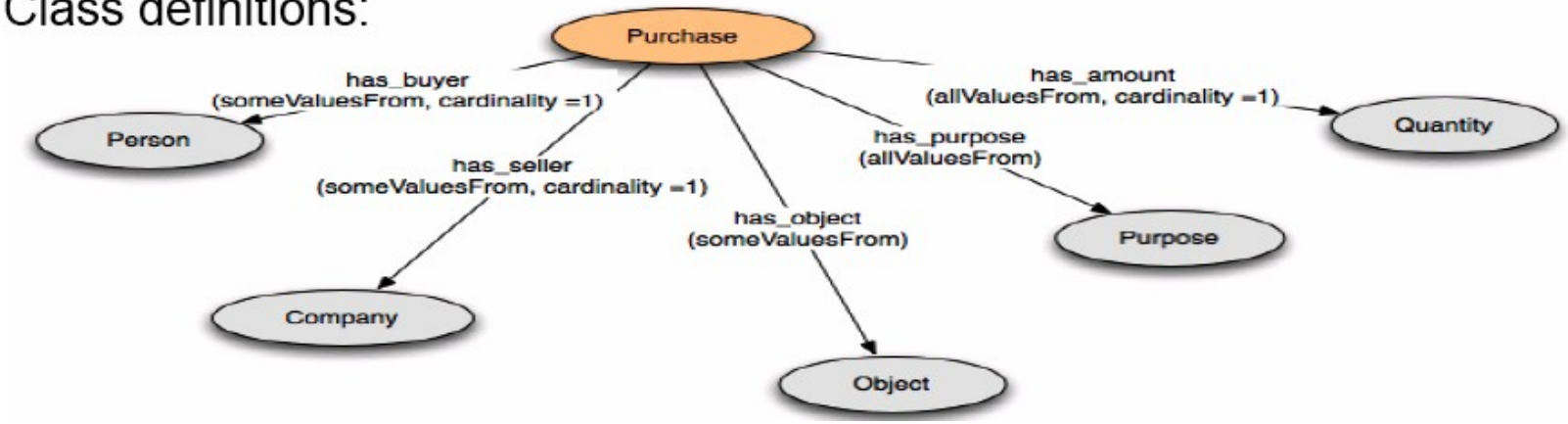
Pattern 1, Example 3

Example: *John buys a "Lenny the Lion" book from books.example.com for $15 as a birthday gift*

- The relation explicitly has more than one participant, and, in many contexts, none of them can be considered a primary one, thus an individual is created to represent the relation instance with links to all participants:



Class definitions:



Considerations in introducing a new class

- We did not give meaningful names to instances of properties or to the classes used to represent instances of n-ary relations, but merely label them.
- In most cases, these individuals do not stand on their own but merely function as auxiliaries to group together other objects. Hence a distinguishing name serves no purpose. Note that a similar approach is taken when reifying statements in RDF.
- Creating a class to represent an n-ary relation limits the use of many OWL constructs and creates a maintenance problem, especially when dealing with inverse relations.

Pattern 2

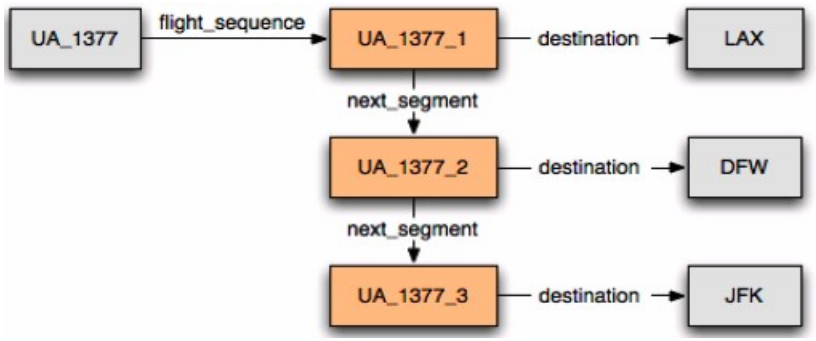
Using lists for arguments in a relation

- Some n-ary relations do not naturally fall into either of the use cases above, but are more similar to a list or sequence of arguments.
- Example: *United Airlines flight 3177 visits the following airports: LAX, DFW, and JFK*
- The relation holds between the flight and the airports it visits, in the order of the arrival of the aircraft at each airport in turn.
- This relation might hold between many different numbers of arguments, and there is no natural way to break it up into a set of distinct properties relating the flight to each airport. The order of the arguments is highly meaningful.

Pattern 2, N-ary relations

Example: *United Airlines flight 3177 visits the following airports: LAX, DFW, and JFK*

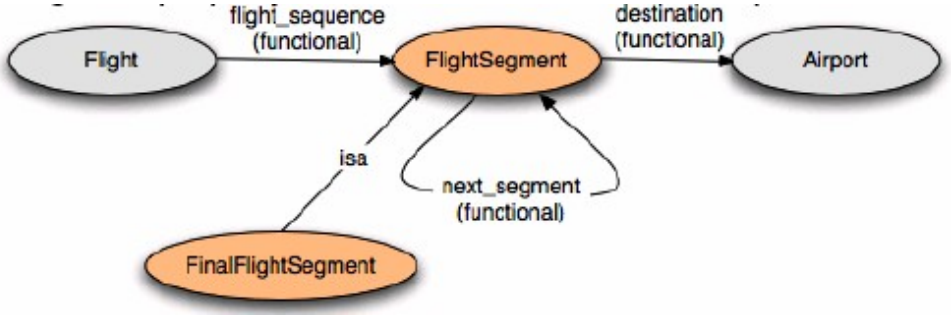
- Basic idea: when all but one participant in a relation do not have a specific role and essentially form an ordered list, it is natural to connect these arguments into a sequence according to some relation, and to relate the one participant to this sequence (or the first element of the sequence)



nextSegment is an ordering relation between instances of the *FlightSegment class*; each flight segment has a property for the destination of that segment

- *A special subclass of flight segment, FinalFlightSegment is added with a maximum cardinality of 0 on the nextSegment property, to indicate the end of the sequence.*

Class definitions:

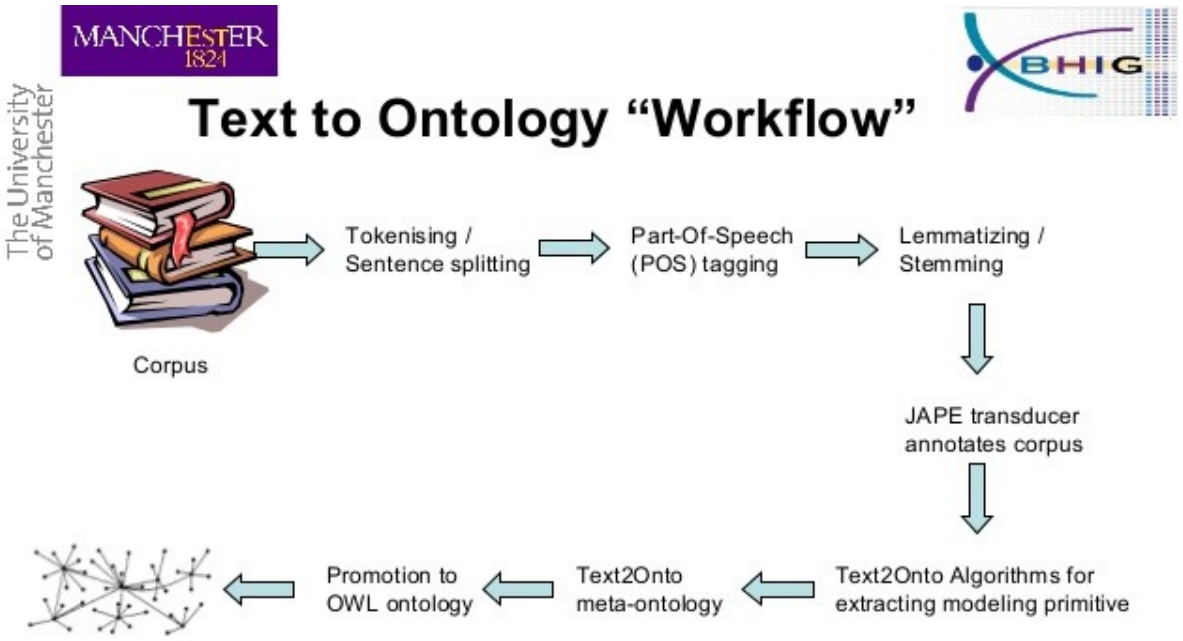


Additional resources

- W3C Working Group Note -Defining N-ary Relations on the Semantic Web
- http://www.w3.org/TR/swbp-n-aryRelations
- W3C Semantic Web Best Practices and Deployment Working Group
- http://www.w3.org/2001/sw/BestPractices/
- General references on Semantic Web
- http://www.w3.org/2001/sw/

+ many other resources/tutorials on the Web

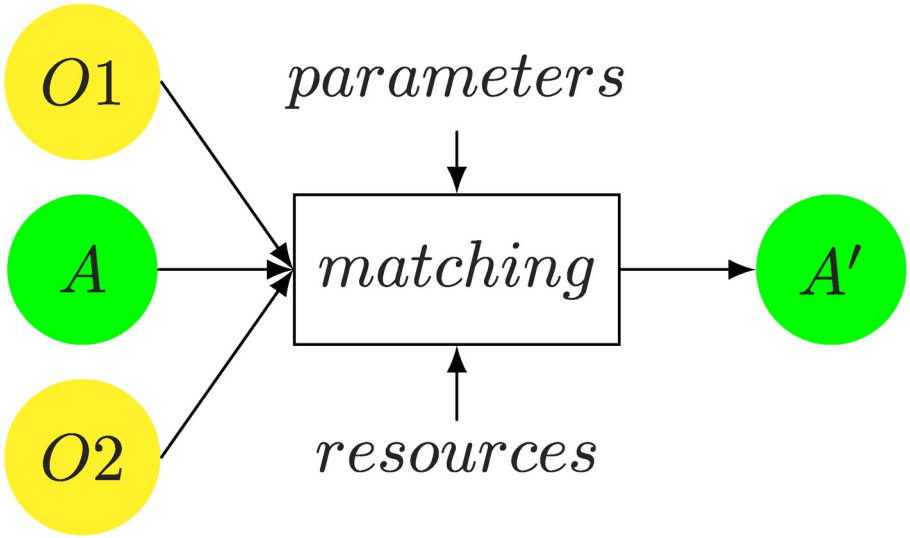
Ontology learning



Tutorial by Icaro Medeiros

https://www.slideshare.net/icaromedeiros/slidesontolearning

Ontology matching



Tutorial by Pavel Shvaiko Jérôme Euzenat

http://disi.unitn.it/~p2p/matching/SWAP06-OMtutorial.pdf

Tools & benchmarcks

http://www.ontologymatching.org/index.html

http://www.mkbergman.com/1769/50-ontology-mapping-and-alignment-tools/

http://oaei.ontologymatching.org/

Some research from here…

Automatic Ontology Matching Via Upper Ontologies: A Systematic Evaluation

Viviana Mascardi, Angela Locoro, Paolo Rosso

Abstract—"Ontology matching" is the process of finding correspondences between entities belonging to different ontologies. This paper describes a set of algorithms that exploit upper ontologies as semantic bridges in the ontology matching process and presents a systematic analysis of the relationships among features of matched ontologies (number of simple and composite concepts, stems, concepts at the top level, common English suffixes and prefixes, ontology depth), matching algorithms, used upper ontologies, and experiment results. This analysis allowed us to state under which circumstances the exploitation of upper ontologies gives significant advantages with respect to traditional approaches that do no use them. We run experiments with SUMO-OWL (a restricted version of SUMO), OpenCyc and DOLCE. The experiments demonstrate that when our "structural matching method via upper ontology" uses an upper ontology large enough (OpenCyc, SUMO-OWL), the recall is significantly improved while preserving the precision obtained without upper ontologies. Instead, our "non structural matching method" via OpenCyc and SUMO-OWL improves the precision and maintains the recall. The "mixed method" that combines the results of structural alignment without using upper ontologies and structural alignment via upper ontologies improves the recall and maintains the F-measure independently of the used upper ontology.

Index Terms—Ontology Matching, Upper Ontology.

2010

Some research from here…

Semantic Web and Declarative Agent Languages and Technologies: Current and Future Trends*
(Position Paper)

2012

Viviana Mascardi[1], James Hendler[2], and Laura Papaleo[3]

[1] DIBRIS, University of Genova, Italy
viviana.mascardi@unige.it
[2] Rensselaer Polytechnic Institute, Troy, NY, USA
hendler@cs.rpi.edu
[3] ICT Department, Provincia di Genova, Genova, Italy
papaleo@disi.unige.it

1 Introduction

One of the first discussions about a Web enriched with semantics and its relationships with artificial intelligence (and hence, with intelligent agents) dates back to 1998 [3], but it was only ten years ago that the idea of a Semantic Web on top of which agent-based computing would have allowed computer programs to interact with non-local web-based resources, became familiar to a wide audience of scientists [4,18].

Some research from here…

Web Intelligence and Agent Systems: An International Journal 5 (2007) 1–5
IOS Press

CooL-AgentSpeak: Endowing AgentSpeak-DL Agents with Plan Exchange and Ontology Services [1]

Viviana Mascardi [a,*], Davide Ancona [a], Matteo Barbieri [a], Rafael H. Bordini [b] and Alessandro Ricci [c]
[a] *DIBRIS – University of Genova*
Via Dodecaneso 35, 16146, Genova, Italy
Email: viviana.mascardi@unige.it, davide.ancona@unige.it, matteo.barbieri@oniriclabs.com
[b] *FACIN – PUCRS, Av. Ipiranga 6681, 90619-900, Porto Alegre - RS, Brazil*
Email: r.bordini@pucrs.br
[c] *DEIS – University of Bologna*
Via Venezia 52, 47023, Cesena (FC), Italy
Email: a.ricci@unibo.it

Abstract. In this paper we present CooL-AgentSpeak, an extension of AgentSpeak-DL with plan exchange and ontology services. In CooL-AgentSpeak, the search for an ontologically relevant plan is no longer limited to the agent's local plan library but is carried out in the other agents' libraries too, according to a cooperation strategy, and it is not based solely on unification and on the subsumption relation between concepts, but also on ontology matching. Belief querying and updating also take advantage of ontological reasoning and matching.

Keywords: AgentSpeak, cooperation, plan exchange, ontology matching

2014

Some research from here…

Wiley Online Library

Search

Agent-oriented and ontology-driven digital libraries: the IndianaMAS experience

Daniela Briola, Vincenzo Deufemia ✉, Viviana Mascardi, Luca Paolino

First published: 13 March 2017 | **https://doi.org/10.1002/spe.2494** | Citations: 2

Read the full text ›

PDF TOOLS SHARE

Summary

This paper describes IndianaMAS, a multiagent system able to automatically classify and manage images, sketches, and multilingual documents in a cultural heritage domain. The latter has been formalized by means of an ontology, which enables the semantic integration of heterogeneous data from different sources, drives the agent communication with the internal and external environment, and provides an abstract and human-readable interface between the system and the user. IndianaMAS is able to expose to the world the classified data via a digital library. Modularity and reusability are the key engineering principles followed in the system design and implementation. We present the details of the IndianaMAS system and discuss how its architecture can be generalized to create – with the minimal effort – systems addressing similar classification, storage, and management problems, but operating in different domains and driven

2017

Some research from here…

OntoScene, A Logic-based Scene Interpreter: Implementation and Application in the Rock Art Domain

DANIELA BRIOLA
Department of Computer Sciences, Systems and Communications
University of Milano Bicocca, Italy
(*e-mail:* daniela.briola@unimib.it)

VIVIANA MASCARDI and MASSIMILIANO GIOSEFFI
Department of Informatics, Bioengineering, Robotics, and Systems Engineering
University of Genova, Italy
(*e-mail:* viviana.mascardi@unige.it, gmaxsun89@gmail.com)

submitted 14 January 2019; revised 29 August 2019; accepted 4 November 2019

Abstract

We present OntoScene, a framework aimed at understanding the semantics of visual scenes starting from the semantics of their elements and the spatial relations holding between them. OntoScene exploits ontologies for representing knowledge and Prolog for specifying the interpretation rules that domain experts may adopt, and for implementing the SceneInterpreter engine. Ontologies allow the designer to formalize the domain in a reusable way, and make the system modular and interoperable with existing multiagent systems, while Prolog provides a solid basis to define complex rules of interpretation in a way that can be affordable even for people with no background in Computational Logics. The domain selected for experimenting

2020

Some research from here…



European Conference on Multi-Agent Systems
↳ EUMAS 2023: **Multi-Agent Systems** pp 66–82 | Cite as

Integrating Ontologies and Cognitive Conversational Agents in On2Conv

Zeinab Namakizadeh Esfahani, Débora Cristina Engelmann, Angelo Ferrando, Massimiliano Margarone & Viviana Mascardi

Conference paper | First Online: 07 September 2023

127 Accesses

Part of the Lecture Notes in Computer Science book series (LNAI,volume 14282)

Abstract

Multiagent systems have been successfully used in many domains. Being social, they are expected to communicate with human users in natural language. Nevertheless, the natural interaction between agents and humans is still challenging. Chatbot technologies are a key enabler to boost the communication between humans and software agents, but few technical

2023

Some research from here…

Integrating Procedural Ontologies in VEsNA: Study, Requirements, and Preliminary Design

Diana Ivan[1,*,†], Angelo Ferrando[2,*,†], Andrea Gatti[1,*,†], Giovanna Guerrini[1,*,†] and Viviana Mascardi[1,*,†]

[1]*University of Genova, Italy*
[2]*University of Modena-Reggio Emilia, Italy*

2024

Abstract

In this paper we overview the literature on procedural ontologies and on the integration of ontologies in cognitive and declarative agent frameworks. Based on what we learn from this study, we make a proposal for representing procedural knowledge as OWL ontologies and we outline an extension of the VEsNA framework to integrate them and take advantage of their features.

Keywords

Procedural ontologies, BDI agents, VEsNA

1. Introduction

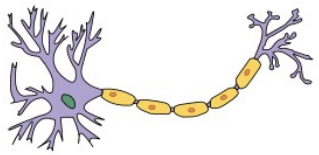
Quoting the widely accepted definition by Jennings, Sycara, and Wooldridge [1], an intelligent agent is a computer system that is *situated* within an environment, is capable of engaging in *social* interactions, and is autonomous, reactive, and proactive. The strong definition of

Applications of Ontologies

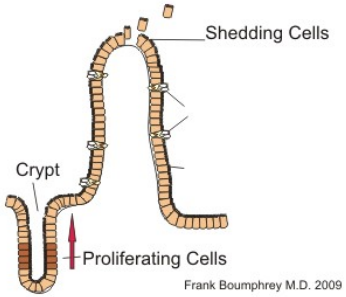
Cell Ontology

The Cell Ontology is a structured controlled vocabulary for cell types in animals.

OntoBee | AberOWL | OLS







ID Space	cl
PURL	http://purl.obolibrary.org/obo/cl.owl
License	CC-BY
Homepage	
Contact	
Trackers	https://github.com/obophenotype/cell-ontology/issues
Domain	cells
TermGenie	http://cl.termgenie.org
Taxon	Metazoa
Dependencies	uberon go ro chebi pato
Jobs	build passing

The Cell Ontology is designed as a structured controlled vocabulary for cell types. This ontology was constructed for use by the model organism and other bioinformatics databases, where there is a need for a controlled vocabulary of cell types. This ontology is not organism specific it covers cell types from prokaryotes to mammals. However, it excludes plant cell types, which are covered by PO.

View | Edit | PURL

Generated by: _layouts/ontology_detail.html

See metadata guide

Edit the metadata for this page: cl.md

(remember to make a fork and pull request!)

Integrates material from the tutorial by Ian Horrocks

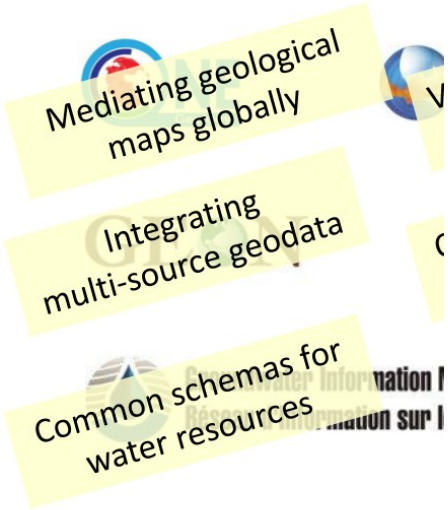
http://slideplayer.com/slide/4915102/

Applications of Ontologies

e-Science, e.g., Geoscience

Applying ontologies in geosciences

Provide novel functions in



Challenges
in applying an ontology spectrum
to promote geological data interoperability

- Modeling vs. encoding
- Multilinguality
- Flexibility & usefulness
- Mediation & evolution

Objective

Address these challenges, and provide a route map for applying an ontology spectrum to promote geological data interoperability

Applications of Ontologies

e-Science, e.g., Geoscience

http://www.geoscience-semantics.org/ontologies/

https://www.w3.org/2003/01/geo/

http://www.geonames.org/ontology/documentation.html

Applications of Ontologies

e-Scien

http://

https:/

http://

Semantic Enhancement of Volunteered Geographic Information

Laura Di Rocco

DIBRIS - Università degli Studi di Genova

Abstract. The continue use of social media has created a revolution in the production of data in terms of heterogeneity and volume. This huge amount of data is often georeferenced, and the geospatial position of data is a very relevant dimension for analysis. The considerable heterogeneity and the variable quality of user-generated data prevent however the full exploitation of this information. In this PhD project, we focus on user-generated geospatial data, commonly referred to as Volunteered Geographic Information, where geographical objects are annotated by using tags. The simplicity to use tags allows users to put a lot of tags on objects. This creates a big and noisy tagspace in which it is hard to find data tagged by other people. To overcome this heterogeneity and quality problem, a solution is to rely on ontologies to classify spatial entities tags and names. This problem is addressed in the PhD project with specific reference to the tags used in OpenStreetMap for describing georeferenced objects relevant in urban context, producing as result an ontology for classifying such objects, and a corresponding ontology population approach.

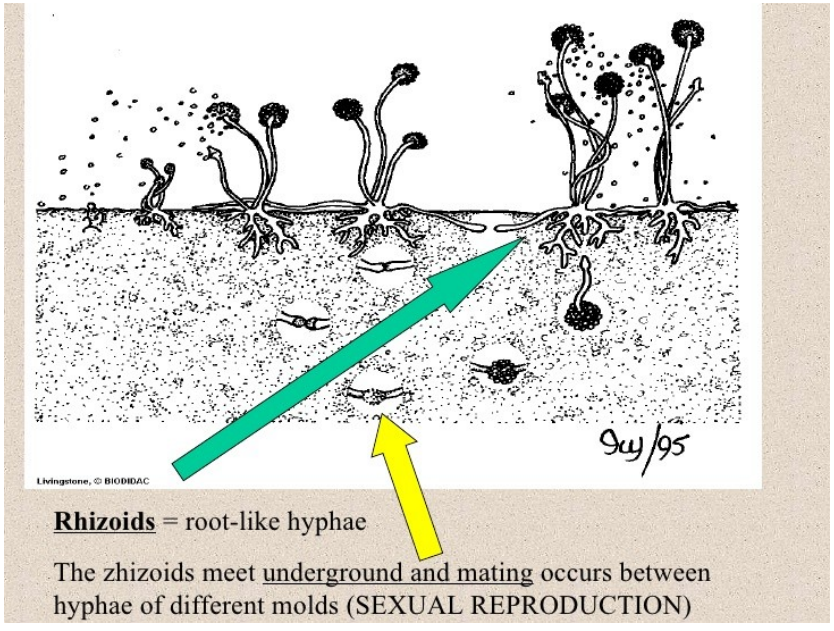
Keywords: Geographic Ontology, Implicit/Explicit Geographical Information, Volunteered Geographical Information, Semantic Classification

Applications of Ontologies

e-Science, e.g., Bioinformatics

Used e.g., for “in silico” investigations relating theory and data

- E.g., relating data on plant structure and functionalities



Applications of Ontologies

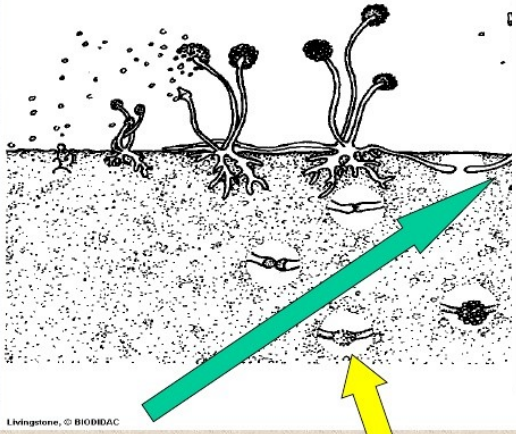
e-Science, e.g., Bioinformatics

Used e.g., for “in silico” investigations relating theory and data

- E.g., re

Classes

- plant anatomical entity
  - plant anatomical space
  - plant structure
    - cardinal part of multi-tissu
    - collective plant structure
    - embryo plant structure
    - in vitro plant structure
    - multi-tissue plant structure
    - plant cell
    - plant ovary
    - portion of plant tissue
    - rhizoid
    - trichome
    - trichome apex
    - trichome tip
    - whole plant
  - portion of plant substance



Rhizoids = root-like hyphae

The zhizoids meet underground and mating occurs between hyphae of different molds (SEXUAL REPRODUCTION)

PO - Plant Ontology

Overview | Browse | DLQuery | Download

Annotation	Value
label	plant structure
definition	An anatomical structure that is or was part of a plant, or was deri part of a plant.
class	http://purl.obolibrary.org/obo/PO_0009011
ontology	PO
SubClassOf	plant anatomical entity
has_obo_namespace	plant_anatomy
id	PO:0009011
comment	'Part' includes both proper parts and the whole plant. CARO:0000 anatomical structure is defined as: Material anatomical entity that 3D shape and is generated by coordinated expression of the orga genome.
database cross reference	PO GIT:57

Applications of Ontologies

e-Science, e.g., Bioinformatics

Books and papers:

Ontologies for Bioinformatics,
https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2735951/

Ontologies for Bioinformatics (Computational Molecular Biology),
https://mitpress.mit.edu/books/ontologies-bioinformatics

Anatomy Ontologies for Bioinformatics, Principles and Practice
http://www.springer.com/la/book/9781846288845

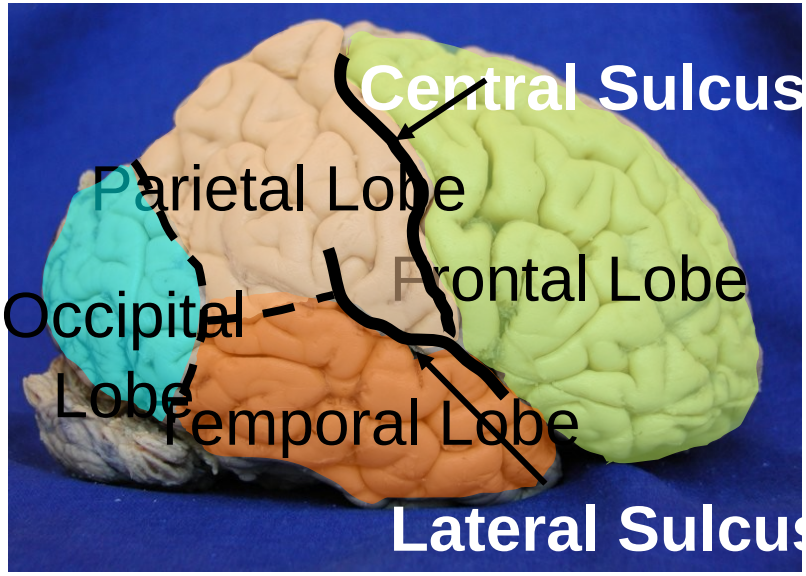
Online resources:

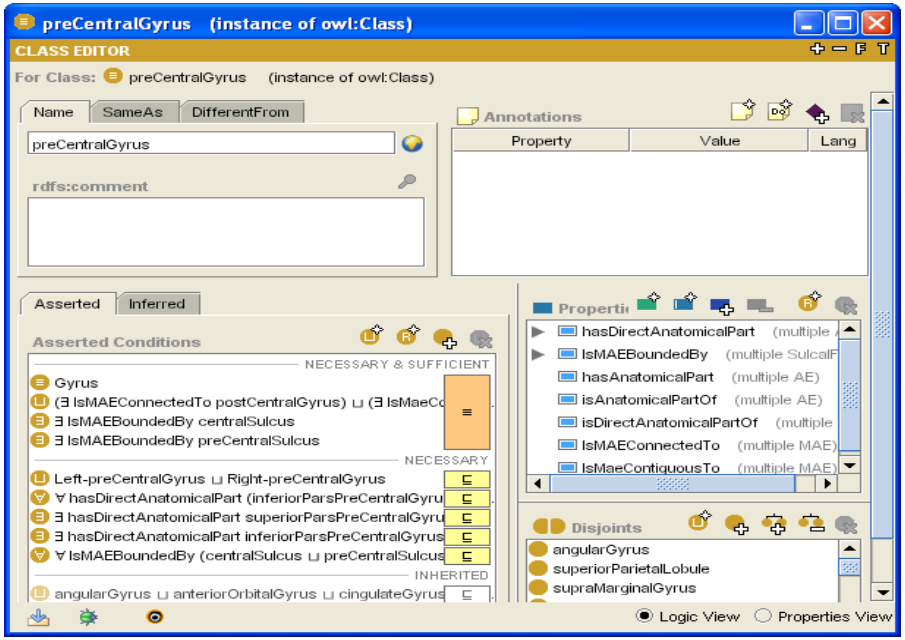
http://www.obofoundry.org/

Applications of Ontologies

Medicine

Building/maintaining terminologies such as Snomed, NCI & Galen

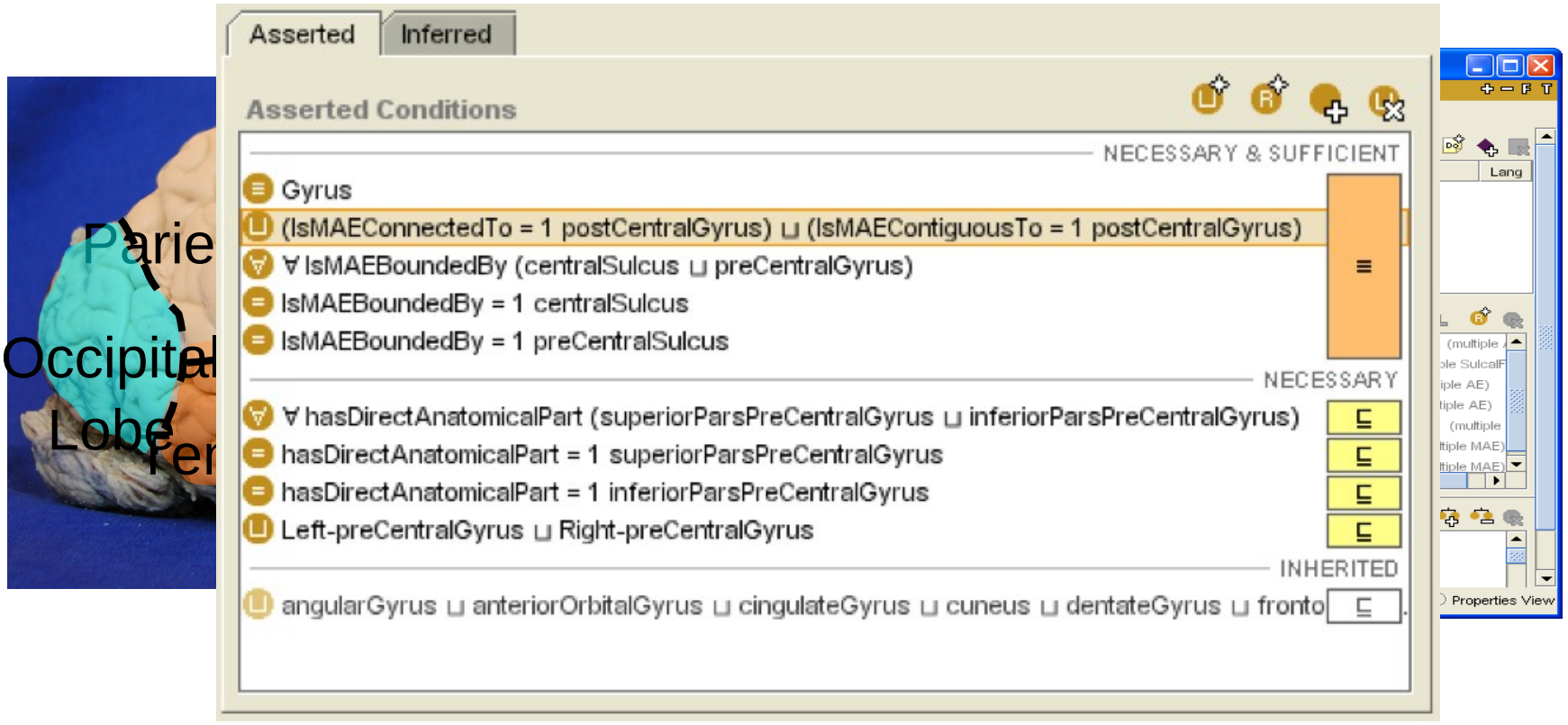




Applications of Ontologies

Medicine

Building/maintaining terminologies such as Snomed, NCI & Galen



Applications of Ontologies

Medicine



Building an Ontological Framework for Healthcare: The Case of the Health Cluster

Keith KT Toh (RMIT University, Australia), Margaret E. Heffernan (RMIT University, Australia), Vass Karpathiou (Deakin University & RMIT University, Australia), Arkalgud Ramaprasad (UIC, USA) and Nilmini Wickramasinghe (Epworth HealthCare, Australia & Deakin University, Australia)

Source Title: Handbook of Research on Healthcare Administration and Management
Copyright: © 2017 | Pages: 16
DOI: 10.4018/978-1-5225-0920-2.ch039

OnDemand PDF Download: $30.00 List Price: $37.50

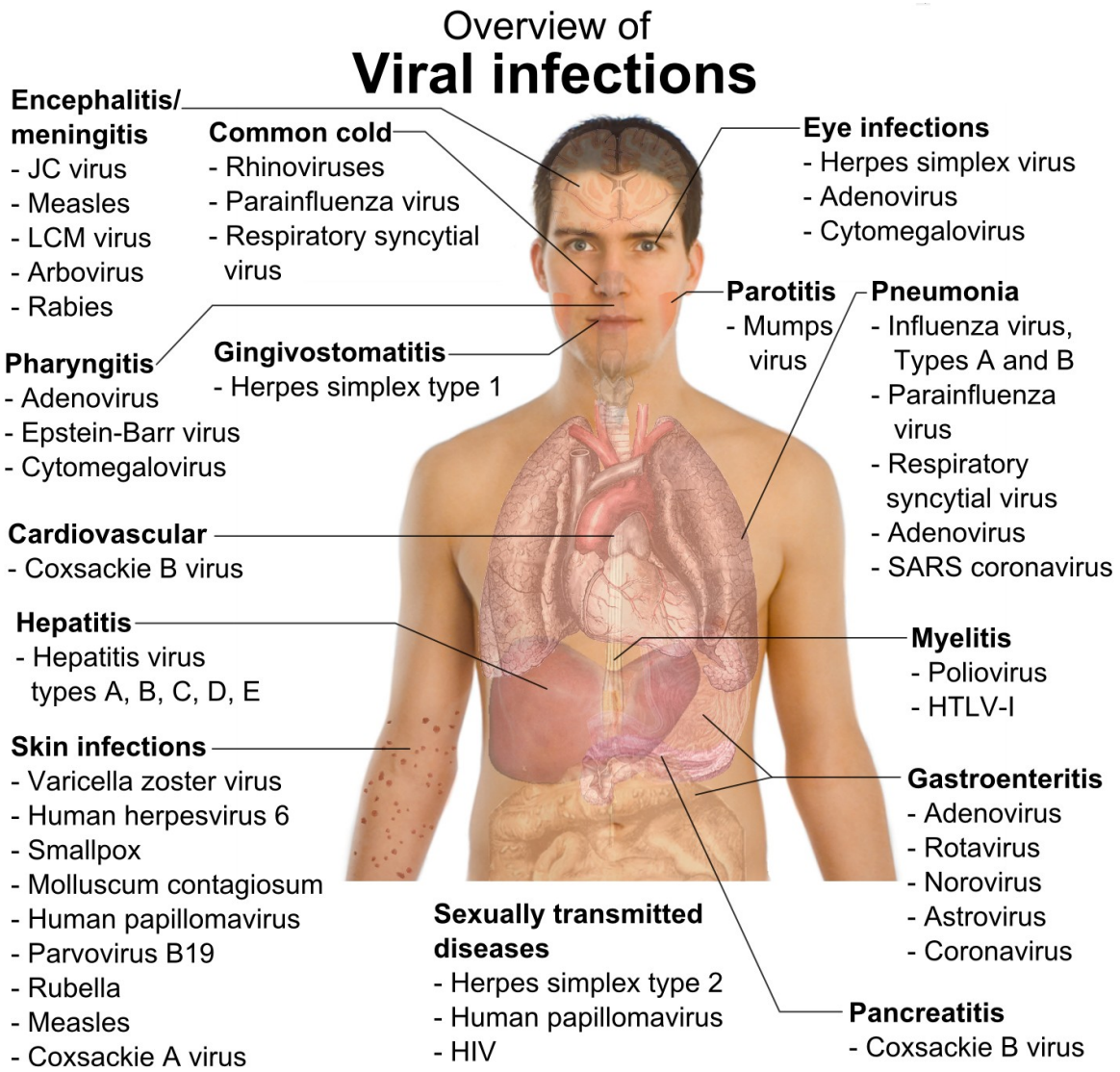
Abstract

Health is a multidisciplinary domain which necessitates experts from diverse backgrounds coming together to effect optimal care delivery for patients. Such a context can benefit by being framed as a knowledge cluster. To illustrate, the case study of a health cluster research group is mapped in terms of ontology. In this way, key relationships and informational exchanges are captured and this in turn can enable more prudent use of critical knowledge assets within the cluster. This ontology is then proffered for the healthcare domain in general as the following discusses.

Applications of Ontologies

Medicine

DISEASE ONTOLOGY



Applications of Ontologies

Medicine



Classes

- bacterial infectious disease
- fungal infectious disease
- parasitic infectious disease
- viral infectious disease
  - Argentine hemorrhagic fe
  - Barmah Forest virus disea
  - Bolivian hemorrhagic feve
  - Brazilian hemorrhagic fev
  - Chapare hemorrhagic feve
  - Colorado tick fever
  - Crimean-Congo hemorrha
  - Eastern equine encephalit
  - Ebola hemorrhagic fever
  - Japanese encephalitis
  - Kyasanur forest disease

DOID - Human Disease Ontology

Overview | Browse | DLQuery | Download

Annotation	Value
label	viral infectious disease
definition	A disease by infectious agent that results_in infection, has_material_basis_in Viruses.
class	http://purl.obolibrary.org/obo/DOID_934
ontology	DOID
Equivalent	disease by infectious agent and (has material basis in some Viruses)
SubClassOf	has material basis in some Viruses, disease by infectious agent
has_obo_namespace	disease_ontology
id	DOID:934

- Rubella
- Measles
- Coxsackie A virus

- Herpes simplex type 2
- Human papillomavirus
- HIV

Pancreatitis
- Coxsackie B virus

Applications of Ontologies

Medicine

http://www.obofoundry.org/ontology/doid.html

https://ehealth.fbk.eu/resources/icpc-2-ontology

Applications of Ontologies

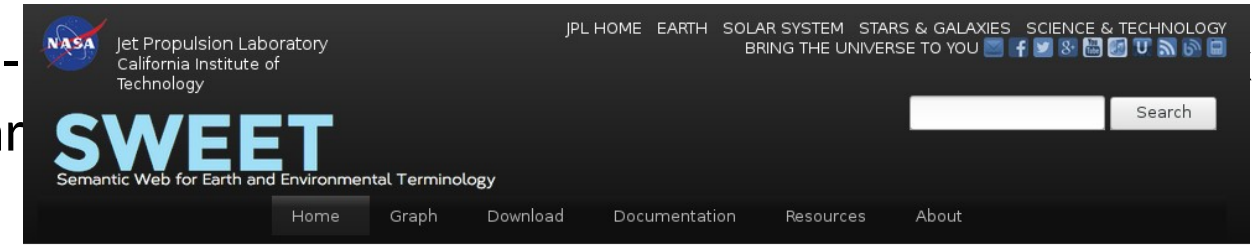
Organising complex and semi-structured information

UN-FAO, NASA, Ordnance Survey, General Motors, Lockheed Martin, …

Applications of Ontologies

Organising complex and semi-structured information

UN- heed
Mar

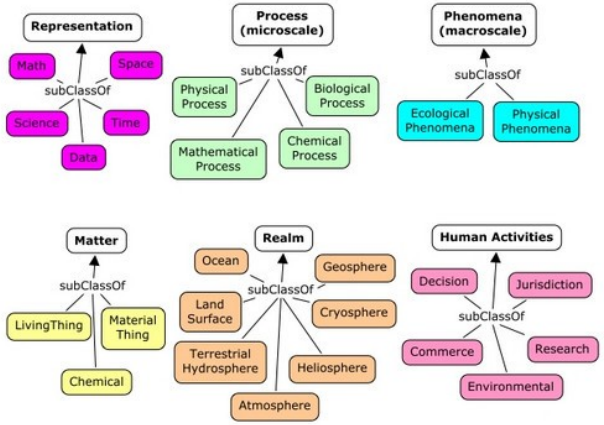


SWEET Overview

SWEET ontologies are written in the OWL ontology language and are publicly available. SWEET 2.3 is highly modular with 6000 concepts in 200 separate ontologies. You can view the entire concept space from an OWL tool such as Protege by reading in sweetAll.owl. Alternatively, these ontologies can be viewed individually. SWEET 2.3 consists of nine top-level concepts/ontologies. Some of the next-level concepts are shown in the Figure. SWEET is a middle-level ontology; most users add a domain-specific ontology using the components defined here to satisfy end user needs.

Interactive Ontologies Graph

Alternatively, click a domain on the image below to automatically visualize a sample subset of related concepts. (Warning: the graph may slow down your computer!)

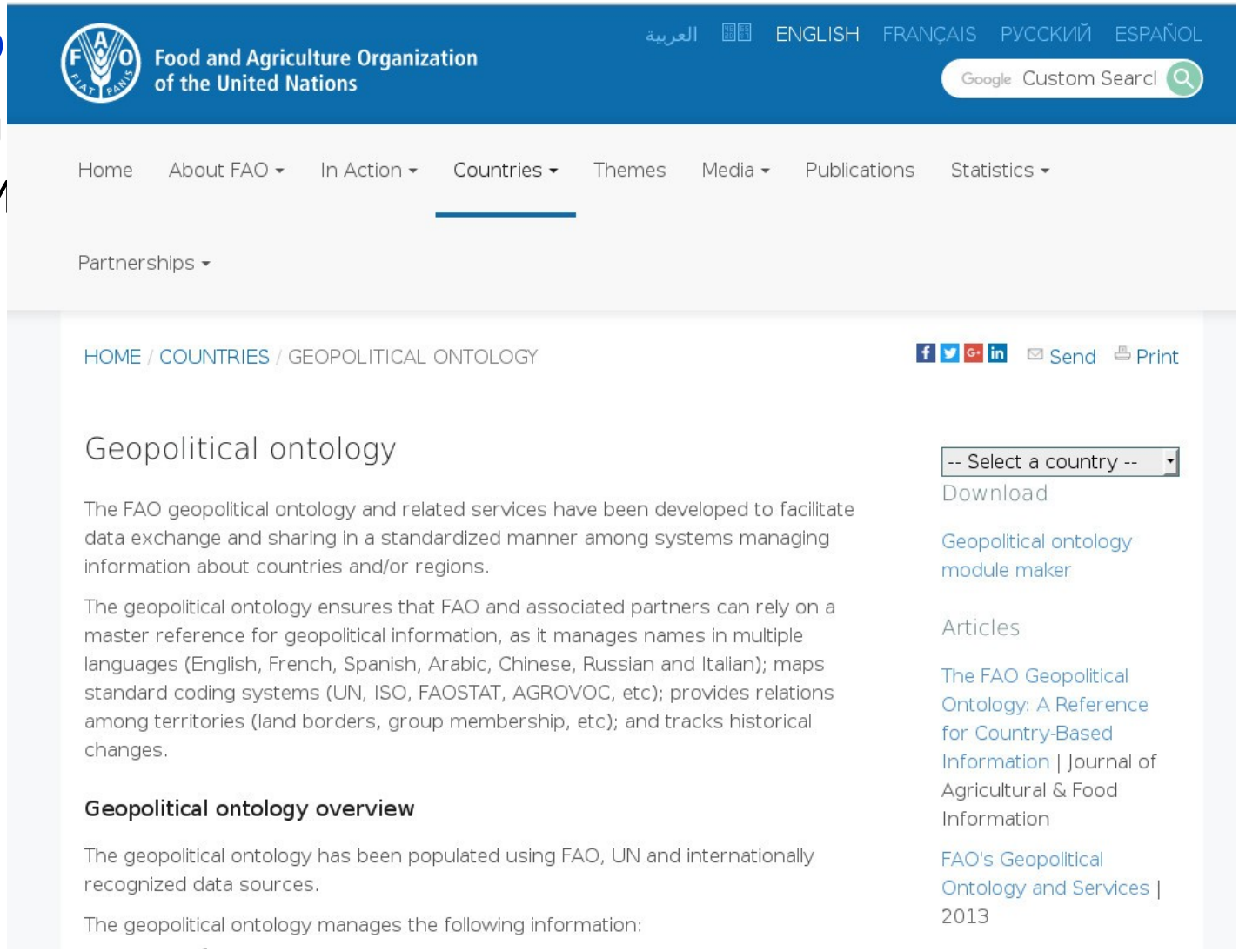


Applications of Ontologies

O

U

M



Applications of Ontologies

Organising complex and semi-structured information

UN- .ockheed Mar



Euro-Mediterranean Conference
EuroMed 2014: Digital Heritage. Progress in Cultural Heritage: Documentation, Preservation, and Protection pp 217-227 | Cite as

Ontology-Driven Processing and Management of Digital Rock Art Objects in IndianaMAS

Authors | Authors and affiliations

Daniela Briola, Vincenzo Deufemia, Viviana Mascardi, Luca Paolino, Nicoletta Bianchi

Conference paper

1 Citations | 5 Readers | 2.3k Downloads

Part of the Lecture Notes in Computer Science book series (LNCS, volume 8740)

Applications of Ontologies

Organising complex and semi-structured information

UN heed
Ma



Intelligent Distributed Computing VI pp 97-102 | Cite as

MUSE: MUltilinguality and SEmantics for the Citizens of the World

Authors Authors and affiliations

Michele Bozzano ✉, Daniela Briola, Diego Leone,

Angela Locoro, Lanfranco Marasso, Viviana Mascardi

Conference paper

566

Downloads

Part of the Studies in Computational Intelligence book series (SCI, volume 446)

Applications of Ontologies

Organisi n

UN-FAO Lockheed Martin, .

Ontologica: Exploiting ontologies and natural language for railway management. Design, implementation and usage examples

Issue title: Selected papers of KES2012 - Part 1 of 2

Guest editors: M. Graña, A.I. Gonzalez-Acuña and C. Zanni-Merk

Article type: Research Article

Authors: Briola, Daniela[a; *] | Caccia, Riccardo[b] | Bozzano, Michele[a] | Locoro, Angela[a; *]

Affiliations: [a] Department of Informatics, Bioengineering, Robotics and System Engineering, University of Genoa, Genova, Italy | [b] Ansaldo STS, a Finmeccanica Company, Genova, Italy | Computational Intelligence Group, University of the Basque Country, UPV/EHU, Spain

Correspondence: [*] Corresponding author: Daniela Briola, Department of Informatics, Bioengineering, Robotics and System Engineering, University of Genoa, Via Opera Pia, 13. 16145 Genova, Italy. Tel.: +39 010 4040449; E-mail: daniela.briola@unige.it. Angela Locoro, Department of Informatics, Bioengineering, Robotics and System Engineering, University of Genoa, Via Opera Pia, 13. 16145 Genova, Italy. Tel.: +39 010 353 6637; E-mail: angela.locoro@unige.it

Abstract: This paper presents the "Ontologica" system, a forefront project born from a joint effort between the Department of Informatics, Bioengineering, Robotics and System Engineering of Genoa university and Ansaldo STS (from the same city) in the design of advanced information systems. The aim of the project is twofold: the adoption of ontologies to manage the Centralized Traffic Control (CTC) logics of a railway system; the improvement of the user interface through the exploitation of natural language queries. Being Ansaldo STS a leader in the railway and metro scope, this project aims to develop a system to be tested on a real and large dataset before being adopted in the railway stations using the CTC system. The first results we obtained are very promising and the system is currently under testing and improvement. In this paper we present the "Ontologica" rationale and architecture with some usage examples.

Keywords: Ontology, natural language processing, knowledge representation

Applications of Ontologies

Military/Government/Legal domains

DARPA, NSA, NIST, SAIC, MoD, Department of Homeland Security, …

http://oegov.us/

https://github.com/RinkeHoekstra/lkif-core

Applications of Ontologies

The Semantic Web



